

3 Processing Raw Text

The most important source of texts is undoubtedly the Web. It's convenient to have existing text collections to explore, such as the corpora we saw in the previous chapters. However, you probably have your own text sources in mind, and need to learn how to access them.

The goal of this chapter is to answer the following questions:

1. How can we write programs to access text from local files and from the web, in order to get hold of an unlimited range of language material?
2. How can we split documents up into individual words and punctuation symbols, so we can carry out the same kinds of analysis we did with text corpora in earlier chapters?
3. How can we write programs to produce formatted output and save it in a file?

In order to address these questions, we will be covering key concepts in NLP, including tokenization and stemming. Along the way you will consolidate your Python knowledge and learn about strings, files, and regular expressions. Since so much text on the web is in HTML format, we will also see how to dispense with markup.

Note

Important: From this chapter onwards, our program samples will assume you begin your interactive session or your program with the following import statements:

```
>>> from __future__ import division
>>> import nltk, re, pprint
```

3.1 Accessing Text from the Web and from Disk

Electronic Books

A small sample of texts from Project Gutenberg appears in the NLTK corpus collection. However, you may be interested in analyzing other texts from Project Gutenberg. You can browse the catalog of 25,000 free online books at <http://www.gutenberg.org/catalog/>, and obtain a URL to an ASCII text file. Although 90% of the texts in Project Gutenberg are in English, it includes material in over 50 other languages, including Catalan, Chinese, Dutch, Finnish, French, German, Italian, Portuguese and Spanish (with more than 100 texts each).

Text number 2554 is an English translation of *Crime and Punishment*, and we can access it as follows.

```
>>> from urllib import urlopen
>>> url = "http://www.gutenberg.org/files/2554/2554.txt"
>>> raw = urlopen(url).read()
>>> type(raw)
<type 'str'>
>>> len(raw)
1176831
>>> raw[:75]
'The Project Gutenberg EBook of Crime and Punishment, by Fyodor Dostoevsky\r\n'
```

Note

The `read()` process will take a few seconds as it downloads this large book. If you're using an internet proxy which is not correctly detected by Python, you may need to specify the proxy manually as follows:

```
>>> proxies = {'http': 'http://www.someproxy.com:3128'}
>>> raw = urlopen(url, proxies=proxies).read()
```

The variable `raw` contains a string with 1,176,831 characters. (We can see that it is a string, using `type(raw)`.) This is the raw content of the book, including many details we are not interested in such as whitespace, line breaks and blank lines. Notice the `\r` and `\n` in the opening line of the file, which is how Python displays the special carriage return and line feed characters (the file must have been created

on a Windows machine). For our language processing, we want to break up the string into words and punctuation, as we saw in [1](#). This step is called **tokenization**, and it produces our familiar structure, a list of words and punctuation.

```
>>> tokens = nltk.word_tokenize(raw)
>>> type(tokens)
<type 'list'>
>>> len(tokens)
255809
>>> tokens[:10]
['The', 'Project', 'Gutenberg', 'EBook', 'of', 'Crime', 'and', 'Punishment', ',', 'by']
```

Notice that NLTK was needed for tokenization, but not for any of the earlier tasks of opening a URL and reading it into a string. If we now take the further step of creating an NLTK text from this list, we can carry out all of the other linguistic processing we saw in [1](#), along with the regular list operations like slicing:

```
>>> text = nltk.Text(tokens)
>>> type(text)
<type 'nltk.text.Text'>
>>> text[1020:1060]
['CHAPTER', 'I', 'On', 'an', 'exceptionally', 'hot', 'evening', 'early', 'in',
'July', 'a', 'young', 'man', 'came', 'out', 'of', 'the', 'garret', 'in',
'which', 'he', 'lodged', 'in', 'S', '.', 'Place', 'and', 'walked', 'slowly',
',', 'as', 'though', 'in', 'hesitation', ',', 'towards', 'K', '.', 'bridge', '.']
>>> text.collocations()
Katerina Ivanovna; Pulcheria Alexandrovna; Avdotya Romanovna; Pyotr
Petrovitch; Project Gutenberg; Marfa Petrovna; Rodion Romanovitch;
Sofya Semyonovna; Nikodim Fomitch; did not; Hay Market; Andrey
Semyonovitch; old woman; Literary Archive; Dmitri Prokofitch; great
deal; United States; Praskovya Pavlovna; Porfiry Petrovitch; ear rings
```

Notice that *Project Gutenberg* appears as a collocation. This is because each text downloaded from Project Gutenberg contains a header with the name of the text, the author, the names of people who scanned and corrected the text, a license, and so on. Sometimes this information appears in a footer at the end of the file. We cannot reliably detect where the content begins and ends, and so have to resort to manual inspection of the file, to discover unique strings that mark the beginning and the end, before trimming `raw` to be just the content and nothing else:

```
>>> raw.find("PART I")
5303
>>> raw.rfind("End of Project Gutenberg's Crime")
1157681
>>> raw = raw[5303:1157681] ❶
>>> raw.find("PART I")
0
```

The `find()` and `rfind()` ("reverse find") methods help us get the right index values to use for slicing the string ❶. We overwrite `raw` with this slice, so now it begins with "PART I" and goes up to (but not including) the phrase that marks the end of the content.

This was our first brush with the reality of the web: texts found on the web may contain unwanted material, and there may not be an automatic way to remove it. But with a small amount of extra work we can extract the material we need.

Dealing with HTML

Much of the text on the web is in the form of HTML documents. You can use a web browser to save a page as text to a local file, then access this as described in the section on files below. However, if you're going to do this often, it's easiest to get Python to do the work directly. The first step is the same as before, using `urlopen`. For fun we'll pick a BBC News story called *Blondes to die out in 200 years*, an urban legend passed along by the BBC as established scientific fact:

```
>>> url = "http://news.bbc.co.uk/2/hi/health/2284783.stm"
>>> html = urlopen(url).read()
>>> html[:60]
'<!doctype html public "-//W3C//DTD HTML 4.0 Transitional//EN'
```

You can type `print html` to see the HTML content in all its glory, including meta tags, an image map, JavaScript, forms, and tables.

Getting text out of HTML is a sufficiently common task that NLTK provides a helper function `nltk.clean_html()`, which takes an HTML string and returns raw text. We can then tokenize this to get our familiar text structure:

```
>>> raw = nltk.clean_html(html)
>>> tokens = nltk.word_tokenize(raw)
>>> tokens
```

```
['BBC', 'NEWS', '|', 'Health', '|', 'Blondes', '"', 'to', 'die', 'out', ...]
```

This still contains unwanted material concerning site navigation and related stories. With some trial and error you can find the start and end indexes of the content and select the tokens of interest, and initialize a text as before.

```
>>> tokens = tokens[96:399]
>>> text = nltk.Text(tokens)
>>> text.concordance('gene')
they say too few people now carry the gene for blondes to last beyond the next tw
t blonde hair is caused by a recessive gene . In order for a child to have blonde
to have blonde hair , it must have the gene on both sides of the family in the gra
there is a disadvantage of having that gene or by chance . They don ' t disappear
ondes would disappear is if having the gene was a disadvantage and I do not think
```

Note

For more sophisticated processing of HTML, use the *Beautiful Soup* package, available from <http://www.crummy.com/software/BeautifulSoup/>

Processing Search Engine Results

The web can be thought of as a huge corpus of unannotated text. Web search engines provide an efficient means of searching this large quantity of text for relevant linguistic examples. The main advantage of search engines is size: since you are searching such a large set of documents, you are more likely to find any linguistic pattern you are interested in. Furthermore, you can make use of very specific patterns, which would only match one or two examples on a smaller example, but which might match tens of thousands of examples when run on the web. A second advantage of web search engines is that they are very easy to use. Thus, they provide a very convenient tool for quickly checking a theory, to see if it is reasonable.

Table 3.1:

Google Hits for Collocations: The number of hits for collocations involving the words *absolutely* or *definitely*, followed by one of *adore*, *love*, *like*, or *prefer*. (Lieberman, in *LanguageLog*, 2005).

Google hits	<i>adore</i>	<i>love</i>	<i>like</i>	<i>prefer</i>
<i>absolutely</i>	289,000	905,000	16,200	644
<i>definitely</i>	1,460	51,000	158,000	62,600
ratio	198:1	18:1	1:10	1:97

Unfortunately, search engines have some significant shortcomings. First, the allowable range of search patterns is severely restricted. Unlike local corpora, where you write programs to search for arbitrarily complex patterns, search engines generally only allow you to search for individual words or strings of words, sometimes with wildcards. Second, search engines give inconsistent results, and can give widely different figures when used at different times or in different geographical regions. When content has been duplicated across multiple sites, search results may be boosted. Finally, the markup in the result returned by a search engine may change unpredictably, breaking any pattern-based method of locating particular content (a problem which is ameliorated by the use of search engine APIs).

Note

Your Turn: Search the web for "the of" (inside quotes). Based on the large count, can we conclude that *the of* is a frequent collocation in English?

Processing RSS Feeds

The blogosphere is an important source of text, in both formal and informal registers. With the help of a third-party Python library called the *Universal Feed Parser*, freely downloadable from <http://feedparser.org/>, we can access the content of a blog, as shown below:

```
>>> import feedparser
>>> llog = feedparser.parse("http://languagelog ldc.upenn.edu/nll/?feed=atom")
>>> llog['feed']['title']
u'Language Log'
>>> len(llog.entries)
15
>>> post = llog.entries[2]
>>> post.title
```

```

u"He's My BF"
>>> content = post.content[0].value
>>> content[:70]
u'<p>Today I was chatting with three of our visiting graduate students f'
>>> nltk.word_tokenize(nltk.clean_html(content))
>>> nltk.word_tokenize(nltk.clean_html(llog.entries[2].content[0].value))
[u'Today', u'I', u'was', u'chatting', u'with', u'three', u'of', u'our', u'visiting',
u'graduate', u'students', u'from', u'the', u'PRC', u'.', u'Thinking', u'that', u'I',
u'was', u'being', u'au', u'courant', u',', u'I', u'mentioned', u'the', u'expression',
u'DUI4XIANG4', u'\u5c0d\u8c61', u('', u'boy', u'/', u'girl', u'friend', u'', ...]
```

Note that the resulting strings have a `u` prefix to indicate that they are Unicode strings (see [3.3](#)). With some further work, we can write programs to create a small corpus of blog posts, and use this as the basis for our NLP work.

Reading Local Files

In order to read a local file, we need to use Python's built-in `open()` function, followed by the `read()` method. Suppose you have a file `document.txt`, you can load its contents like this:

```

>>> f = open('document.txt')
>>> raw = f.read()
```

Note

Your Turn: Create a file called `document.txt` using a text editor, and type in a few lines of text, and save it as plain text. If you are using IDLE, select the *New Window* command in the *File* menu, typing the required text into this window, and then saving the file as `document.txt` inside the directory that IDLE offers in the pop-up dialogue box. Next, in the Python interpreter, open the file using `f = open('document.txt')`, then inspect its contents using `print f.read()`.

Various things might have gone wrong when you tried this. If the interpreter couldn't find your file, you would have seen an error like this:

```

>>> f = open('document.txt')
Traceback (most recent call last):
  File "<pyshell#7>", line 1, in -toplevel-
    f = open('document.txt')
IOError: [Errno 2] No such file or directory: 'document.txt'
```

To check that the file that you are trying to open is really in the right directory, use IDLE's *Open* command in the *File* menu; this will display a list of all the files in the directory where IDLE is running. An alternative is to examine the current directory from within Python:

```

>>> import os
>>> os.listdir('.')
```

Another possible problem you might have encountered when accessing a text file is the newline conventions, which are different for different operating systems. The built-in `open()` function has a second parameter for controlling how the file is opened: `open('document.txt', 'rU')` — '`r`' means to open the file for reading (the default), and '`U`' stands for "Universal", which lets us ignore the different conventions used for marking newlines.

Assuming that you can open the file, there are several methods for reading it. The `read()` method creates a string with the contents of the entire file:

```

>>> f.read()
'Time flies like an arrow.\nFruit flies like a banana.\n'
```

Recall that the `'\n'` characters are **newlines**; this is equivalent to pressing *Enter* on a keyboard and starting a new line.

We can also read a file one line at a time using a `for` loop:

```

>>> f = open('document.txt', 'rU')
>>> for line in f:
...     print line.strip()
Time flies like an arrow.
Fruit flies like a banana.
```

Here we use the `strip()` method to remove the newline character at the end of the input line.

NLTK's corpus files can also be accessed using these methods. We simply have to use `nltk.data.find()` to get the filename for any corpus item. Then we can open and read it in the way we just demonstrated above:

```
>>> path = nltk.data.find('corpora/gutenberg/melville-moby_dick.txt')
>>> raw = open(path, 'rU').read()
```

Extracting Text from PDF, MSWord and other Binary Formats

ASCII text and HTML text are human readable formats. Text often comes in binary formats — like PDF and MSWord — that can only be opened using specialized software. Third-party libraries such as `pypdf` and `pywin32` provide access these formats. Extracting text from multi-column documents is particularly challenging. For once-off conversion of a few documents, it is simpler to open the document with a suitable application, then save it as text to your local drive, and access it as described below. If the document is already on the web, you can enter its URL in Google's search box. The search result often includes a link to an HTML version of the document, which you can save as text.

Capturing User Input

Sometimes we want to capture the text that a user inputs when she is interacting with our program. To prompt the user to type a line of input, call the Python function `raw_input()`. After saving the input to a variable, we can manipulate it just as we have done for other strings.

```
>>> s = raw_input("Enter some text: ")
Enter some text: On an exceptionally hot evening early in July
>>> print "You typed", len(nltk.word_tokenize(s)), "words."
You typed 8 words.
```

The NLP Pipeline

[3.1](#) summarizes what we have covered in this section, including the process of building a vocabulary that we saw in [1](#). (One step, normalization, will be discussed in [3.6](#)).

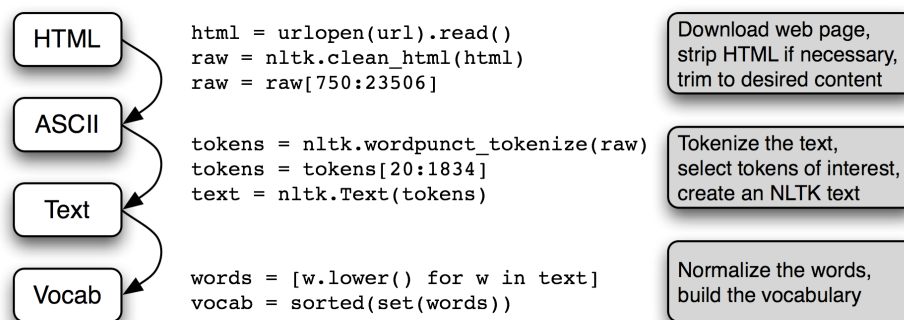


Figure 3.1: The Processing Pipeline: We open a URL and read its HTML content, remove the markup and select a slice of characters; this is then tokenized and optionally converted into an `nltk.Text` object; we can also lowercase all the words and extract the vocabulary.

There's a lot going on in this pipeline. To understand it properly, it helps to be clear about the type of each variable that it mentions. We find out the type of any Python object `x` using `type(x)`, e.g. `type(1)` is `<int>` since `1` is an integer.

When we load the contents of a URL or file, and when we strip out HTML markup, we are dealing with strings, Python's `<str>` data type (We will learn more about strings in [3.2](#)):

```
>>> raw = open('document.txt').read()
>>> type(raw)
<type 'str'>
```

When we tokenize a string we produce a list (of words), and this is Python's `<list>` type. Normalizing and sorting lists produces other lists:

```
>>> tokens = nltk.word_tokenize(raw)
>>> type(tokens)
<type 'list'>
```

```
>>> words = [w.lower() for w in tokens]
>>> type(words)
<type 'list'>
>>> vocab = sorted(set(words))
>>> type(vocab)
<type 'list'>
```

The type of an object determines what operations you can perform on it. So, for example, we can append to a list but not to a string:

```
>>> vocab.append('blog')
>>> raw.append('blog')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'str' object has no attribute 'append'
```

Similarly, we can concatenate strings with strings, and lists with lists, but we cannot concatenate strings with lists:

```
>>> query = 'Who knows?'
>>> beatles = ['john', 'paul', 'george', 'ringo']
>>> query + beatles
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot concatenate 'str' and 'list' objects
```

3.2 Strings: Text Processing at the Lowest Level

It's time to study a fundamental data type that we've been studiously avoiding so far. In earlier chapters we focused on a text as a list of words. We didn't look too closely at words and how they are handled in the programming language. By using NLTK's corpus interface we were able to ignore the files that these texts had come from. The contents of a word, and of a file, are represented by programming languages as a fundamental data type known as a **string**. In this section we explore strings in detail, and show the connection between strings, words, texts and files.

Basic Operations with Strings

Strings are specified using single quotes ^❶ or double quotes ^❷, as shown below. If a string contains a single quote, we must backslash-escape the quote ^❸ so Python knows a literal quote character is intended, or else put the string in double quotes ^❷. Otherwise, the quote inside the string ^❹ will be interpreted as a close quote, and the Python interpreter will report a syntax error:

```
>>> monty = 'Monty Python' ❶
>>> monty
'Monty Python'
>>> circus = "Monty Python's Flying Circus" ❷
>>> circus
'Monty Python's Flying Circus'
>>> circus = 'Monty Python\'s Flying Circus' ❸
>>> circus
'Monty Python's Flying Circus'
>>> circus = 'Monty Python's Flying Circus' ❹
  File "<stdin>", line 1
    circus = 'Monty Python's Flying Circus'
                ^
SyntaxError: invalid syntax
```

Sometimes strings go over several lines. Python provides us with various ways of entering them. In the next example, a sequence of two strings is joined into a single string. We need to use backslash ^❶ or parentheses ^❷ so that the interpreter knows that the statement is not complete after the first line.

```
>>> couplet = "Shall I compare thee to a Summer's day?"\
...           "Thou are more lovely and more temperate:" ❶
>>> print couplet
Shall I compare thee to a Summer's day?Thou are more lovely and more temperate:
>>> couplet = ("Rough winds do shake the darling buds of May,"
...           "And Summer's lease hath all too short a date:") ❷
>>> print couplet
Rough winds do shake the darling buds of May,And Summer's lease hath all too short a date:
```

Unfortunately the above methods do not give us a newline between the two lines of the sonnet. Instead, we can use a triple-quoted string as follows:

```
>>> couplet = """Shall I compare thee to a Summer's day?
... Thou are more lovely and more temperate:"""
>>> print couplet
Shall I compare thee to a Summer's day?
Thou are more lovely and more temperate:
>>> couplet = '''Rough winds do shake the darling buds of May,
... And Summer's lease hath all too short a date:'''
>>> print couplet
Rough winds do shake the darling buds of May,
And Summer's lease hath all too short a date:
```

Now that we can define strings, we can try some simple operations on them. First let's look at the + operation, known as **concatenation** ❶. It produces a new string that is a copy of the two original strings pasted together end-to-end. Notice that concatenation doesn't do anything clever like insert a space between the words. We can even multiply strings ❷:

```
>>> 'very' + 'very' + 'very' ❶
'veryveryvery'
>>> 'very' * 3 ❷
'veryveryvery'
```

Note

Your Turn: Try running the following code, then try to use your understanding of the string + and * operations to figure out how it works. Be careful to distinguish between the string ' ', which is a single whitespace character, and '', which is the empty string.

```
>>> a = [1, 2, 3, 4, 5, 6, 7, 6, 5, 4, 3, 2, 1]
>>> b = [' ' * 2 * (7 - i) + 'very' * i for i in a]
>>> for line in b:
...     print line
```

We've seen that the addition and multiplication operations apply to strings, not just numbers. However, note that we cannot use subtraction or division with strings:

```
>>> 'very' - 'y'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for -: 'str' and 'str'
>>> 'very' / 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for /: 'str' and 'int'
```

These error messages are another example of Python telling us that we have got our data types in a muddle. In the first case, we are told that the operation of subtraction (i.e., -) cannot apply to objects of type str (strings), while in the second, we are told that division cannot take str and int as its two operands.

Printing Strings

So far, when we have wanted to look at the contents of a variable or see the result of a calculation, we have just typed the variable name into the interpreter. We can also see the contents of a variable using the `print` statement:

```
>>> print monty
Monty Python
```

Notice that there are no quotation marks this time. When we inspect a variable by typing its name in the interpreter, the interpreter prints the Python representation of its value. Since it's a string, the result is quoted. However, when we tell the interpreter to `print` the contents of the variable, we don't see quotation characters since there are none inside the string.

The `print` statement allows us to display more than one item on a line in various ways, as shown below:

```
>>> grail = 'Holy Grail'
>>> print monty + grail
Monty PythonHoly Grail
>>> print monty, grail
Monty Python Holy Grail
```



```
>>> print monty, "and the", grail
Monty Python and the Holy Grail
```

Accessing Individual Characters

As we saw in [1.2](#) for lists, strings are indexed, starting from zero. When we index a string, we get one of its characters (or letters). A single character is nothing special — it's just a string of length 1.

```
>>> monty[0]
'M'
>>> monty[3]
't'
>>> monty[5]
' '
```

As with lists, if we try to access an index that is outside of the string we get an error:

```
>>> monty[20]
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
IndexError: string index out of range
```

Again as with lists, we can use negative indexes for strings, where -1 is the index of the last character ^❶. Positive and negative indexes give us two ways to refer to any position in a string. In this case, when the string had a length of 12, indexes 5 and -7 both refer to the same character (a space). (Notice that $5 = \text{len}(\text{monty}) - 7$.)

```
>>> monty[-1] ❶
'n'
>>> monty[5]
' '
>>> monty[-7]
' '
```

We can write `for` loops to iterate over the characters in strings. This `print` statement ends with a trailing comma, which is how we tell Python not to print a newline at the end.

```
>>> sent = 'colorless green ideas sleep furiously'
>>> for char in sent:
...     print char,
...
c o l o r l e s s   g r e e n   i d e a s   s l e e p   f u r i o u s l y
```

We can count individual characters as well. We should ignore the case distinction by normalizing everything to lowercase, and filter out non-alphabetic characters:

```
>>> from nltk.corpus import gutenberg
>>> raw = gutenberg.raw('melville-moby_dick.txt')
>>> fdist = nltk.FreqDist(ch.lower() for ch in raw if ch.isalpha())
>>> fdist.keys()
['e', 't', 'a', 'o', 'n', 'i', 's', 'h', 'r', 'l', 'd', 'u', 'm', 'c', 'w',
 'f', 'g', 'p', 'b', 'y', 'v', 'k', 'q', 'j', 'x', 'z']
```

This gives us the letters of the alphabet, with the most frequently occurring letters listed first (this is quite complicated and we'll explain it more carefully below). You might like to visualize the distribution using `fdist.plot()`. The relative character frequencies of a text can be used in automatically identifying the language of the text.

Accessing Substrings

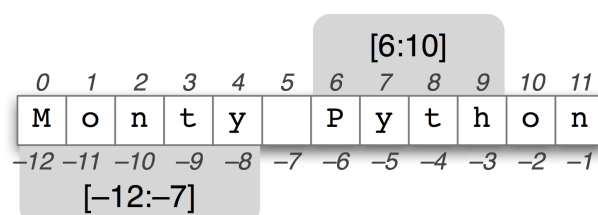


Figure 3.2: String Slicing: The string "Monty Python" is shown along with its positive and negative indexes; two substrings are selected using "slice" notation. The slice `[m,n]` contains the characters from position `m` through `n-1`.

A substring is any continuous section of a string that we want to pull out for further processing. We can easily access substrings using the same slice notation we used for lists (see [3.2](#)). For example, the following code accesses the substring starting at index 6, up to (but not including) index 10:

```
>>> monty[6:10]
'Pyth'
```

Here we see the characters are 'P', 'y', 't', and 'h' which correspond to `monty[6] ... monty[9]` but not `monty[10]`. This is because a slice *starts* at the first index but finishes *one before* the end index.

We can also slice with negative indexes — the same basic rule of starting from the start index and stopping one before the end index applies; here we stop before the space character.

```
>>> monty[-12:-7]
'Monty'
```

As with list slices, if we omit the first value, the substring begins at the start of the string. If we omit the second value, the substring continues to the end of the string:

```
>>> monty[:5]
'Monty'
>>> monty[6:]
'Python'
```

We test if a string contains a particular substring using the `in` operator, as follows:

```
>>> phrase = 'And now for something completely different'
>>> if 'thing' in phrase:
...     print 'found "thing"'
found "thing"
```

We can also find the position of a substring within a string, using `find()`:

```
>>> monty.find('Python')
6
```

Note

Your Turn: Make up a sentence and assign it to a variable, e.g. `sent = 'my sentence...'`. Now write slice expressions to pull out individual words. (This is obviously not a convenient way to process the words of a text!)

More operations on strings

Python has comprehensive support for processing strings. A summary, including some operations we haven't seen yet, is shown in [3.2](#). For more information on strings, type `help(str)` at the Python prompt.

Table 3.2:

Useful String Methods: operations on strings in addition to the string tests shown in [1.4](#); all methods produce a new string or list

Method	Functionality
<code>s.find(t)</code>	index of first instance of string <code>t</code> inside <code>s</code> (-1 if not found)
<code>s.rfind(t)</code>	index of last instance of string <code>t</code> inside <code>s</code> (-1 if not found)
<code>s.index(t)</code>	like <code>s.find(t)</code> except it raises <code>ValueError</code> if not found
<code>s.rindex(t)</code>	like <code>s.rfind(t)</code> except it raises <code>ValueError</code> if not found
<code>s.join(text)</code>	combine the words of the text into a string using <code>s</code> as the glue
<code>s.split(t)</code>	split <code>s</code> into a list wherever a <code>t</code> is found (whitespace by default)
<code>s.splitlines()</code>	split <code>s</code> into a list of strings, one per line
<code>s.lower()</code>	a lowercased version of the string <code>s</code>
<code>s.upper()</code>	an uppercased version of the string <code>s</code>
<code>s.title()</code>	a titlecased version of the string <code>s</code>
<code>s.strip()</code>	a copy of <code>s</code> without leading or trailing whitespace
<code>s.replace(t, u)</code>	replace instances of <code>t</code> with <code>u</code> inside <code>s</code>

The Difference between Lists and Strings

Strings and lists are both kinds of **sequence**. We can pull them apart by indexing and slicing them, and we can join them together by concatenating them. However, we cannot join strings and lists:

```
>>> query = 'Who knows?'
>>> beatles = ['John', 'Paul', 'George', 'Ringo']
>>> query[2]
'o'
>>> beatles[2]
'George'
>>> query[:2]
'Wh'
>>> beatles[:2]
['John', 'Paul']
>>> query + " I don't"
'Who knows? I don't'
>>> beatles + 'Brian'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate list (not "str") to list
>>> beatles + ['Brian']
['John', 'Paul', 'George', 'Ringo', 'Brian']
```

When we open a file for reading into a Python program, we get a string corresponding to the contents of the whole file. If we use a **for** loop to process the elements of this string, all we can pick out are the individual characters — we don't get to choose the granularity. By contrast, the elements of a list can be as big or small as we like: for example, they could be paragraphs, sentences, phrases, words, characters. So lists have the advantage that we can be flexible about the elements they contain, and correspondingly flexible about any downstream processing. Consequently, one of the first things we are likely to do in a piece of NLP code is tokenize a string into a list of strings (3.7). Conversely, when we want to write our results to a file, or to a terminal, we will usually format them as a string (3.9).

Lists and strings do not have exactly the same functionality. Lists have the added power that you can change their elements:

```
>>> beatles[0] = "John Lennon"
>>> del beatles[-1]
>>> beatles
['John Lennon', 'Paul', 'George']
```

On the other hand if we try to do that with a *string* — changing the 0th character in query to 'F' — we get:

```
>>> query[0] = 'F'
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
TypeError: object does not support item assignment
```

This is because strings are **immutable** — you can't change a string once you have created it. However, lists are **mutable**, and their contents can be modified at any time. As a result, lists support operations that modify the original value rather than producing a new value.

Note

Your Turn: Consolidate your knowledge of strings by trying some of the exercises on strings at the end of this chapter.

3.3 Text Processing with Unicode

Our programs will often need to deal with different languages, and different character sets. The concept of "plain text" is a fiction. If you live in the English-speaking world you probably use ASCII, possibly without realizing it. If you live in Europe you might use one of the extended Latin character sets, containing such characters as "ø" for Danish and Norwegian, "ő" for Hungarian, "ñ" for Spanish and Breton, and "ň" for Czech and Slovak. In this section, we will give an overview of how to use Unicode for processing texts that use non-ASCII character sets.

What is Unicode?

Unicode supports over a million characters. Each character is assigned a number, called a **code point**. In Python, code points are written

in the form `\uXXXX`, where `XXXX` is the number in 4-digit hexadecimal form.

Within a program, we can manipulate Unicode strings just like normal strings. However, when Unicode characters are stored in files or displayed on a terminal, they must be encoded as a stream of bytes. Some encodings (such as ASCII and Latin-2) use a single byte per code point, so they can only support a small subset of Unicode, enough for a single language. Other encodings (such as UTF-8) use multiple bytes and can represent the full range of Unicode characters.

Text in files will be in a particular encoding, so we need some mechanism for translating it into Unicode — translation into Unicode is called **decoding**. Conversely, to write out Unicode to a file or a terminal, we first need to translate it into a suitable encoding — this translation out of Unicode is called **encoding**, and is illustrated in [3.3](#).

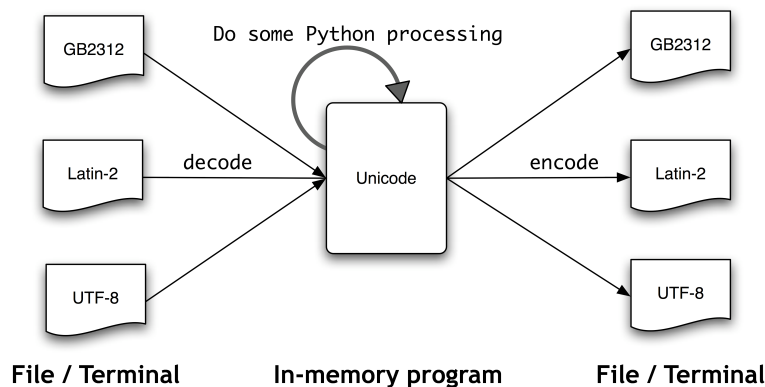


Figure 3.3: Unicode Decoding and Encoding

From a Unicode perspective, characters are abstract entities which can be realized as one or more **glyphs**. Only glyphs can appear on a screen or be printed on paper. A font is a mapping from characters to glyphs.

Extracting encoded text from files

Let's assume that we have a small text file, and that we know how it is encoded. For example, `polish-lat2.txt`, as the name suggests, is a snippet of Polish text (from the Polish Wikipedia; see http://pl.wikipedia.org/wiki/Biblioteka_Pruska). This file is encoded as Latin-2, also known as ISO-8859-2. The function `nltk.data.find()` locates the file for us.

```
>>> path = nltk.data.find('corpora/unicode_samples/polish-lat2.txt')
```

The Python `codecs` module provides functions to read encoded data into Unicode strings, and to write out Unicode strings in encoded form. The `codecs.open()` function takes an encoding parameter to specify the encoding of the file being read or written. So let's import the `codecs` module, and call it with the encoding `'latin2'` to open our Polish file as Unicode.

```
>>> import codecs
>>> f = codecs.open(path, encoding='latin2')
```

For a list of encoding parameters allowed by `codecs`, see <http://docs.python.org/lib/standard-encodings.html>. Note that we can write unicode-encoded data to a file using `f = codecs.open(path, 'w', encoding='utf-8')`.

Text read from the file object `f` will be returned in Unicode. As we pointed out earlier, in order to view this text on a terminal, we need to encode it, using a suitable encoding. The Python-specific encoding `unicode_escape` is a dummy encoding that converts all non-ASCII characters into their `\uXXXX` representations. Code points above the ASCII 0-127 range but below 256 are represented in the two-digit form `\xXX`.

```
>>> for line in f:
...     line = line.strip()
...     print line.encode('unicode_escape')
Pruska Biblioteka Pa\u0144stwowa. Jej dawne zbiory znane pod nazw\u0105
"Berlinka" to skarb kultury i sztuki niemieckiej. Przewiezione przez
Niemc\u00f3w pod koniec II wojny \u015bwiatowej na Dolny \u015ala\u0105sk, zosta\u0142y
odnalezione po 1945 r. na terytorium Polski. Trafi\u0142y do Biblioteki
Jagiello\u0144skiej w Krakowie, obejmuj\u0105 ponad 500 tys. zabytkowych
archiwali\u00f3w, m.in. manuskrypty Goethego, Mozarta, Beethovena, Bacha.
```

The first line above illustrates a Unicode escape string preceded by the `\u` escape string, namely `\u0144`. The relevant Unicode character will be displayed on the screen as the glyph `ń`. In the third line of the preceding example, we see `\xf3`, which corresponds to the glyph `ó`, and is within the 128-255 range.

In Python, a Unicode string literal can be specified by preceding an ordinary string literal with a `u`, as in `u'hello'`. Arbitrary Unicode characters are defined using the `\uXXXX` escape sequence inside a Unicode string literal. We find the integer ordinal of a character using `ord()`. For example:

```
>>> ord('a')
97
```

The hexadecimal 4 digit notation for 97 is 0061, so we can define a Unicode string literal with the appropriate escape sequence:

```
>>> a = u'\u0061'
>>> a
u'a'
>>> print a
a
```

Notice that the Python `print` statement is assuming a default encoding of the Unicode character, namely ASCII. However, `ń` is outside the ASCII range, so cannot be printed unless we specify an encoding. In the following example, we have specified that `print` should use the `repr()` of the string, which outputs the UTF-8 escape sequences (of the form `\xXX`) rather than trying to render the glyphs.

```
>>> nacute = u'\u0144'
>>> nacute
u'\u0144'
>>> nacute_utf = nacute.encode('utf8')
>>> print repr(nacute_utf)
'\xc5\x84'
```

If your operating system and locale are set up to render UTF-8 encoded characters, you ought to be able to give the Python command `print nacute_utf` and see `ń` on your screen.

Note

There are many factors determining what glyphs are rendered on your screen. If you are sure that you have the correct encoding, but your Python code is still failing to produce the glyphs you expected, you should also check that you have the necessary fonts installed on your system.

The module `unicodedata` lets us inspect the properties of Unicode characters. In the following example, we select all characters in the third line of our Polish text outside the ASCII range and print their UTF-8 escaped value, followed by their code point integer using the standard Unicode convention (i.e., prefixing the hex digits with `u+`), followed by their Unicode name.

```
>>> import unicodedata
>>> lines = codecs.open(path, encoding='latin2').readlines()
>>> line = lines[2]
>>> print line.encode('unicode_escape')
Niemc\xfb3w pod koniec II wojny \u015bwiatowej na Dolny \u015al\u0105sk, zosta\u0142y\n
>>> for c in line:
...     if ord(c) > 127:
...         print '%r U+%04x %s' % (c.encode('utf8'), ord(c), unicodedata.name(c))
'\xc3\xfb' U+00fb LATIN SMALL LETTER O WITH ACUTE
'\xc5\x9b' U+015b LATIN SMALL LETTER S WITH ACUTE
'\xc5\x9a' U+015a LATIN CAPITAL LETTER S WITH ACUTE
'\xc4\x85' U+0105 LATIN SMALL LETTER A WITH OGONEK
'\xc5\x82' U+0142 LATIN SMALL LETTER L WITH STROKE
```

If you replace the `%r` (which yields the `repr()` value) by `%s` in the format string of the code sample above, and if your system supports UTF-8, you should see an output like the following:

```
ó U+00f3 LATIN SMALL LETTER O WITH ACUTE
ś U+015b LATIN SMALL LETTER S WITH ACUTE
Ś U+015a LATIN CAPITAL LETTER S WITH ACUTE
ą U+0105 LATIN SMALL LETTER A WITH OGONEK
ł U+0142 LATIN SMALL LETTER L WITH STROKE
```

Alternatively, you may need to replace the encoding `'utf8'` in the example by `'latin2'`, again depending on the details of your system.

The next examples illustrate how Python string methods and the `re` module accept Unicode strings.

```
>>> line.find(u'zosta\u0142y')
54
>>> line = line.lower()
>>> print line.encode('unicode_escape')
niemc\x3f3w pod koniec ii wojny \u015bwiatowej na dolny \u015bl\u0105sk, zosta\u0142y\n
>>> import re
>>> m = re.search(u'\u015b\w*', line)
>>> m.group()
u'\u015bwiatowej'
```

NLTK tokenizers allow Unicode strings as input, and correspondingly yield Unicode strings as output.

```
>>> nltk.word_tokenize(line)
[u'niemc\u015bwiatowej', u'pod', u'koniec', u'ii', u'wojny', u'\u015bwiatowej',
u'na', u'dolny', u'\u015bl\u0105sk', u'zosta\u0142y']
```

Using your local encoding in Python

If you are used to working with characters in a particular local encoding, you probably want to be able to use your standard methods for inputting and editing strings in a Python file. In order to do this, you need to include the string `'# -*- coding: <coding> -*-'` as the first or second line of your file. Note that `<coding>` has to be a string like `'latin-1'`, `'big5'` or `'utf-8'` (see [3.4](#)).

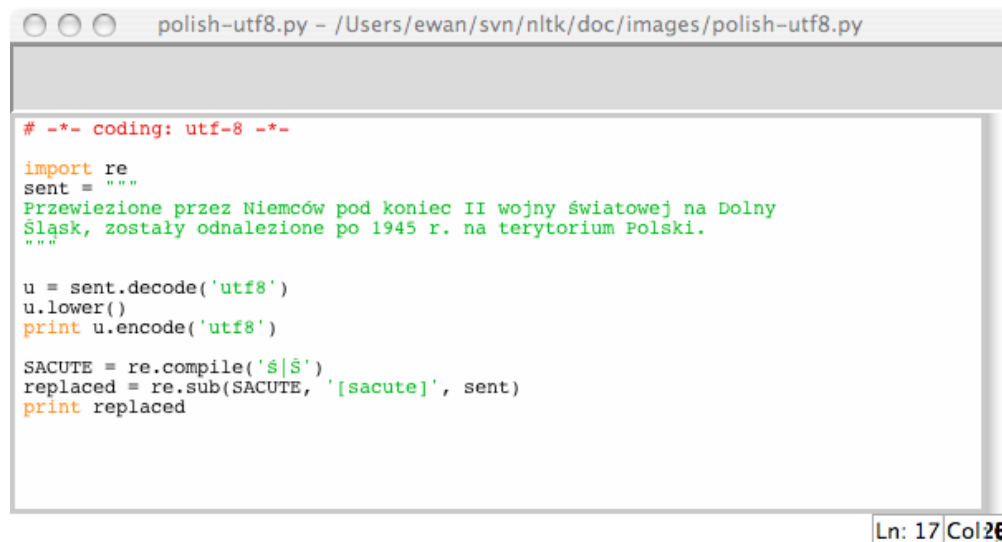


Figure 3.4: Unicode and IDLE: UTF-8 encoded string literals in the IDLE editor; this requires that an appropriate font is set in IDLE's preferences; here we have chosen Courier CE.

The above example also illustrates how regular expressions can use encoded strings.

3.4 Regular Expressions for Detecting Word Patterns

Many linguistic processing tasks involve pattern matching. For example, we can find words ending with *ed* using `endswith('ed')`. We saw a variety of such "word tests" in [1.4](#). Regular expressions give us a more powerful and flexible method for describing the character patterns we are interested in.

Note

There are many other published introductions to regular expressions, organized around the syntax of regular expressions and applied to searching text files. Instead of doing this again, we focus on the use of regular expressions at different stages of linguistic processing. As usual, we'll adopt a problem-based approach and present new features only as they are needed to solve practical problems. In our discussion we will mark regular expressions using chevrons like this: `< patt >`.

To use regular expressions in Python we need to import the `re` library using: `import re`. We also need a list of words to search; we'll use the Words Corpus again ([2.4](#)). We will preprocess it to remove any proper names.

```
>>> import re
>>> wordlist = [w for w in nltk.corpus.words.words('en') if w.islower()]
```

Using Basic Meta-Characters

Let's find words ending with *ed* using the regular expression «*ed*\$». We will use the `re.search(p, s)` function to check whether the pattern `p` can be found somewhere inside the string `s`. We need to specify the characters of interest, and use the dollar sign which has a special behavior in the context of regular expressions in that it matches the end of the word:

```
>>> [w for w in wordlist if re.search('ed$', w)]
['abaised', 'abandoned', 'abased', 'abashed', 'abatised', 'abed', 'aborted', ...]
```

The `.` **wildcard** symbol matches any single character. Suppose we have room in a crossword puzzle for an 8-letter word with *j* as its third letter and *t* as its sixth letter. In place of each blank cell we use a period:

```
>>> [w for w in wordlist if re.search('^..j..t..$', w)]
['abjectly', 'adjuster', 'dejected', 'dejectly', 'injector', 'majestic', ...]
```

Note

Your Turn: The caret symbol `^` matches the start of a string, just like the `$` matches the end. What results do we get with the above example if we leave out both of these, and search for «`..j..t..`»?

Finally, the `?` symbol specifies that the previous character is optional. Thus «`^e-?mail$`» will match both *email* and *e-mail*. We could count the total number of occurrences of this word (in either spelling) in a text using `sum(1 for w in text if re.search('^e-?mail$', w))`.

Ranges and Closures



Figure 3.5: T9: Text on 9 Keys

The **T9** system is used for entering text on mobile phones (see 3.5). Two or more words that are entered with the same sequence of keystrokes are known as **textonyms**. For example, both *hole* and *golf* are entered by pressing the sequence 4653. What other words could be produced with the same sequence? Here we use the regular expression «`^[ghi][mno][jlk][def]$`»:

```
>>> [w for w in wordlist if re.search('^[ghi][mno][jlk][def]$', w)]
['gold', 'golf', 'hold', 'hole']
```

The first part of the expression, «`^[ghi]`», matches the start of a word followed by *g*, *h*, or *i*. The next part of the expression, «`[mno]`», constrains the second character to be *m*, *n*, or *o*. The third and fourth characters are also constrained. Only four words satisfy all these constraints. Note that the order of characters inside the square brackets is not significant, so we could have written «`^[hig][nom][ljk][fed]$`» and matched the same words.

Note

Your Turn: Look for some "finger-twisters", by searching for words that only use part of the number-pad. For example «`^[ghijklmno]+$`», or more concisely, «`^[g-o]+$`», will match words that only use keys 4, 5, 6 in the center row, and «`^[a-fj-o]+$`» will match words that use keys 2, 3, 5, 6 in the top-right corner. What do `-` and `+` mean?

Let's explore the `+` symbol a bit further. Notice that it can be applied to individual letters, or to bracketed sets of letters:

```
>>> chat_words = sorted(set(w for w in nltk.corpus.nps_chat.words()))
>>> [w for w in chat_words if re.search('^+i+n+e+$', w)]
['iiiiiiiiiiiiiiinnnnnnnnnnneeeeeeeeee', 'iiiiiiinnnnnnnnnnneeeeeeee', 'mine',
'mmmmmmmmmiiiiiiiinnnnnnnnnnneeeeeeee']
>>> [w for w in chat_words if re.search('^([ha])+$', w)]
['a', 'aaaaaaaaaaaaaaaaaa', 'aaahhhh', 'ah', 'ahah', 'ahahah', 'ahh',
'ahhahahaha', 'ahhh', 'ahhhh', 'ahhhhhh', 'ahhhhhhhhhhhhhhh', 'h', 'ha', 'haaa',
'hah', 'haha', 'hahaaa', 'hahah', 'hahaha', 'hahaaaa', 'hahahah', 'hahahaha', ...]
```

It should be clear that + simply means "one or more instances of the preceding item", which could be an individual character like m, a set like [fed] or a range like [d-f]. Now let's replace + with * which means "zero or more instances of the preceding item". The regular expression «^m*i*n*e*\$» will match everything that we found using «^m+i+n+e+\$», but also words where some of the letters don't appear at all, e.g. *me*, *min*, and *mmmmm*. Note that the + and * symbols are sometimes referred to as **Kleene closures**, or simply **closures**.

The ^ operator has another function when it appears as the first character inside square brackets. For example «[^][^aeiouAEIOU]» matches any character other than a vowel. We can search the NPS Chat Corpus for words that are made up entirely of non-vowel characters using «[^][^aeiouAEIOU]+\$» to find items like these: :) :), grrr, cyb3r and zzzzzzz. Notice this includes non-alphabetic characters.

Here are some more examples of regular expressions being used to find tokens that match a particular pattern, illustrating the use of some new symbols: `\`, `{}`, `()`, and `|`:

```
>>> wsj = sorted(set(nltk.corpus.treebank.words()))
>>> [w for w in wsj if re.search('^([0-9]+\.[0-9]+$', w)]
['0.0085', '0.05', '0.1', '0.16', '0.2', '0.25', '0.28', '0.3', '0.4', '0.5',
'0.50', '0.54', '0.56', '0.60', '0.7', '0.82', '0.84', '0.9', '0.95', '0.99',
'1.01', '1.1', '1.125', '1.14', '1.1650', '1.17', '1.18', '1.19', '1.2', ...]
>>> [w for w in wsj if re.search('^([A-Z]+\$', w)]
['C$', 'US$']
>>> [w for w in wsj if re.search('^([0-9]{4}$', w)]
['1614', '1637', '1787', '1901', '1903', '1917', '1925', '1929', '1933', ...]
>>> [w for w in wsj if re.search('^([0-9]+-[a-z]{3,5}$', w)]
['10-day', '10-lap', '10-year', '100-share', '12-point', '12-year', ...]
>>> [w for w in wsj if re.search('^([a-z]{5})-([a-z]{2,3})-[a-z]{6}$', w)]
['black-and-white', 'bread-and-butter', 'father-in-law', 'machine-gun-toting',
'savings-and-loan']
>>> [w for w in wsj if re.search('(ed|ing)$', w)]
['62%-owned', 'Absorbed', 'According', 'Adopting', 'Advanced', 'Advancing', ...]
```

Note

Your Turn: Study the above examples and try to work out what the $\setminus$, $\{\}$, $()$, and $|$ notations mean before you read on.

You probably worked out that a backslash means that the following character is deprived of its special powers and must literally match a specific character in the word. Thus, while `.` is special, `\.` only matches a period. The braced expressions, like `{3,5}`, specify the number of repeats of the previous item. The pipe character indicates a choice between the material on its left or its right. Parentheses indicate the scope of an operator: they can be used together with the pipe (or disjunction) symbol like this: `<w(i|e|ai|oo)t>`, matching *wit*, *wet*, *wait*, and *woot*. It is instructive to see what happens when you omit the parentheses from the last expression above, and search for `<ed|ing$>`.

The meta-characters we have seen are summarized in 3.3.

Table 3.3:

Basic Regular Expression Meta-Characters, Including Wildcards, Ranges and Closures

Operator	Behavior
.	Wildcard, matches any character
^abc	Matches some pattern <i>abc</i> at the start of a string
abc\$	Matches some pattern <i>abc</i> at the end of a string
[abc]	Matches one of a set of characters
[A-Z0-9]	Matches one of a range of characters
ed ing s	Matches one of the specified strings (disjunction)
*	Zero or more of previous item, e.g. a^* , $[a-z]^*$ (also known as <i>Kleene Closure</i>)
+	One or more of previous item, e.g. a^+ , $[a-z]^+$
?	Zero or one of the previous item (i.e. optional), e.g. $a?$, $[a-z]?$
{n}	Exactly n repeats where n is a non-negative integer

{n,}	At least n repeats
{,n}	No more than n repeats
{m,n}	At least m and no more than n repeats
a(b c)+	Parentheses that indicate the scope of the operators

To the Python interpreter, a regular expression is just like any other string. If the string contains a backslash followed by particular characters, it will interpret these specially. For example `\b` would be interpreted as the backspace character. In general, when using regular expressions containing backslash, we should instruct the interpreter not to look inside the string at all, but simply to pass it directly to the `re` library for processing. We do this by prefixing the string with the letter `r`, to indicate that it is a **raw string**. For example, the raw string `r'\band\b'` contains two `\b` symbols that are interpreted by the `re` library as matching word boundaries instead of backspace characters. If you get into the habit of using `r'...'` for regular expressions — as we will do from now on — you will avoid having to think about these complications.

3.5 Useful Applications of Regular Expressions

The above examples all involved searching for words `w` that match some regular expression `regex` using `re.search(regex, w)`. Apart from checking if a regular expression matches a word, we can use regular expressions to extract material from words, or to modify words in specific ways.

Extracting Word Pieces

The `re.findall()` ("find all") method finds all (non-overlapping) matches of the given regular expression. Let's find all the vowels in a word, then count them:

```
>>> word = 'supercalifragilisticexpialidocious'
>>> re.findall(r'[aeiou]', word)
['u', 'e', 'a', 'i', 'a', 'i', 'i', 'i', 'e', 'i', 'a', 'i', 'o', 'i', 'o', 'u']
>>> len(re.findall(r'[aeiou]', word))
16
```

Let's look for all sequences of two or more vowels in some text, and determine their relative frequency:

```
>>> wsj = sorted(set(nltk.corpus.treebank.words()))
>>> fd = nltk.FreqDist(vs for word in wsj
...                   for vs in re.findall(r'[aeiou]{2,}', word))
>>> fd.items()
[('io', 549), ('ea', 476), ('ie', 331), ('ou', 329), ('ai', 261), ('ia', 253),
 ('ee', 217), ('oo', 174), ('ua', 109), ('au', 106), ('ue', 105), ('ui', 95),
 ('ei', 86), ('oi', 65), ('oa', 59), ('eo', 39), ('iou', 27), ('eu', 18), ...]
```

Note

Your Turn: In the W3C Date Time Format, dates are represented like this: 2009-12-31. Replace the `?` in the following Python code with a regular expression, in order to convert the string `'2009-12-31'` to a list of integers `[2009, 12, 31]`:

```
[int(n) for n in re.findall(?, '2009-12-31')]
```

Doing More with Word Pieces

Once we can use `re.findall()` to extract material from words, there's interesting things to do with the pieces, like glue them back together or plot them.

It is sometimes noted that English text is highly redundant, and it is still easy to read when word-internal vowels are left out. For example, *declaration* becomes *dclrtm*, and *inalienable* becomes *inlnble*, retaining any initial or final vowel sequences. The regular expression in our next example matches initial vowel sequences, final vowel sequences, and all consonants; everything else is ignored. This three-way disjunction is processed left-to-right, if one of the three parts matches the word, any later parts of the regular expression are ignored. We use `re.findall()` to extract all the matching pieces, and `''.join()` to join them together (see [3.9](#) for more about the join operation).

```
>>> regexp = r'^[AEIOUaeiou]+|[AEIOUaeiou]+$|^[^AEIOUaeiou]
>>> def compress(word):
...     pieces = re.findall(regexp, word)
...     return ''.join(pieces)
```

```

...
>>> english_udhr = nltk.corpus.udhr.words('English-Latin1')
>>> print nltk.tokenwrap(compress(w) for w in english_udhr[:75])
Unvrs1 Dclrtn of Hmn Rghts Prmble Whrs rcgntn of the inhrnt dgnty and
of the eql and inlnble rghts of all mmbrs of the hmn fmly is the fndtn
of frdm , jstce and pce in the wrld , Whrs dsgrd and cntmpt fr hmn
rghts hve rsldt in brbrs acts whch hve outrgd the cnsncne of mnknd ,
and the advnt of a wrld in whch hmn bngs shll enjy frdm of spch and

```

Next, let's combine regular expressions with conditional frequency distributions. Here we will extract all consonant-vowel sequences from the words of Rotokas, such as *ka* and *si*. Since each of these is a pair, it can be used to initialize a conditional frequency distribution. We then tabulate the frequency of each pair:

```

>>> rotokas_words = nltk.corpus.toolbox.words('rotokas.dic')
>>> cvs = [cv for w in rotokas_words for cv in re.findall(r'[ptksvr][aeiou]', w)]
>>> cfd = nltk.ConditionalFreqDist(cvs)
>>> cfd.tabulate()

```

	a	e	i	o	u
k	418	148	94	420	173
p	83	31	105	34	51
r	187	63	84	89	79
s	0	0	100	2	1
t	47	8	0	148	37
v	93	27	105	48	49

Examining the rows for *s* and *t*, we see they are in partial "complementary distribution", which is evidence that they are not distinct phonemes in the language. Thus, we could conceivably drop *s* from the Rotokas alphabet and simply have a pronunciation rule that the letter *t* is pronounced *s* when followed by *i*. (Note that the single entry having *su*, namely *kasuari*, 'cassowary' is borrowed from English).

If we want to be able to inspect the words behind the numbers in the above table, it would be helpful to have an index, allowing us to quickly find the list of words that contains a given consonant-vowel pair, e.g. `cv_index['su']` should give us all words containing *su*. Here's how we can do this:

```

>>> cv_word_pairs = [(cv, w) for w in rotokas_words
...                  for cv in re.findall(r'[ptksvr][aeiou]', w)]
>>> cv_index = nltk.Index(cv_word_pairs)
>>> cv_index['su']
['kasuari']
>>> cv_index['po']
['kaapo', 'kaapopato', 'kaipori', 'kaiporipie', 'kaiporivira', 'kapo', 'kapoa',
'kapokao', 'kapokapo', 'kapokapo', 'kapokapo', 'kapokapo', 'kapokapora', ...]

```

This program processes each word *w* in turn, and for each one, finds every substring that matches the regular expression `«[ptksvr][aeiou]»`. In the case of the word *kasuari*, it finds *ka*, *su* and *ri*. Therefore, the `cv_word_pairs` list will contain `('ka', 'kasuari')`, `('su', 'kasuari')` and `('ri', 'kasuari')`. One further step, using `nltk.Index()`, converts this into a useful index.

Finding Word Stems

When we use a web search engine, we usually don't mind (or even notice) if the words in the document differ from our search terms in having different endings. A query for *laptops* finds documents containing *laptop* and vice versa. Indeed, *laptop* and *laptops* are just two forms of the same dictionary word (or lemma). For some language processing tasks we want to ignore word endings, and just deal with word stems.

There are various ways we can pull out the stem of a word. Here's a simple-minded approach which just strips off anything that looks like a suffix:

```

>>> def stem(word):
...     for suffix in ['ing', 'ly', 'ed', 'ious', 'ies', 'ive', 'es', 's', 'ment']:
...         if word.endswith(suffix):
...             return word[:-len(suffix)]
...     return word

```

Although we will ultimately use NLTK's built-in stemmers, it's interesting to see how we can use regular expressions for this task. Our first step is to build up a disjunction of all the suffixes. We need to enclose it in parentheses in order to limit the scope of the disjunction.

```

>>> re.findall(r'^.*(ing|ly|ed|ious|ies|ive|es|s|ment)$', 'processing')
['ing']

```

Here, `re.findall()` just gave us the suffix even though the regular expression matched the entire word. This is because the parentheses

have a second function, to select substrings to be extracted. If we want to use the parentheses to specify the scope of the disjunction, but not to select the material to be output, we have to add `?:`, which is just one of many arcane subtleties of regular expressions. Here's the revised version.

```
>>> re.findall(r'^.(?:ing|ly|ed|ious|ies|ive|es|s|ment)$', 'processing')
['processing']
```

However, we'd actually like to split the word into stem and suffix. So we should just parenthesize both parts of the regular expression:

```
>>> re.findall(r'^.(*)(ing|ly|ed|ious|ies|ive|es|s|ment)$', 'processing')
(['process', 'ing'])
```

This looks promising, but still has a problem. Let's look at a different word, *processes*:

```
>>> re.findall(r'^.(*)(ing|ly|ed|ious|ies|ive|es|s|ment)$', 'processes')
(['processe', 's'])
```

The regular expression incorrectly found an *-s* suffix instead of an *-es* suffix. This demonstrates another subtlety: the star operator is "greedy" and the `.*` part of the expression tries to consume as much of the input as possible. If we use the "non-greedy" version of the star operator, written `*?`, we get what we want:

```
>>> re.findall(r'^.(?*)(ing|ly|ed|ious|ies|ive|es|s|ment)$', 'processes')
(['process', 'es'])
```

This works even when we allow an empty suffix, by making the content of the second parentheses optional:

```
>>> re.findall(r'^.(?*)(ing|ly|ed|ious|ies|ive|es|s|ment)?$', 'language')
(['language', ''])
```

This approach still has many problems (can you spot them?) but we will move on to define a function to perform stemming, and apply it to a whole text:

```
>>> def stem(word):
...     regexp = r'^.(?*)(ing|ly|ed|ious|ies|ive|es|s|ment)?$'
...     stem, suffix = re.findall(regexp, word)[0]
...     return stem
...
>>> raw = """DENNIS: Listen, strange women lying in ponds distributing swords
... is no basis for a system of government. Supreme executive power derives from
... a mandate from the masses, not from some farcical aquatic ceremony."""
>>> tokens = nltk.word_tokenize(raw)
>>> [stem(t) for t in tokens]
['DENNIS', ':', 'Listen', ',', 'strange', 'women', 'ly', 'in', 'pond',
'distribut', 'sword', 'i', 'no', 'basis', 'for', 'a', 'system', 'of', 'govern',
',', 'Supreme', 'execut', 'power', 'deriv', 'from', 'a', 'mandate', 'from',
'the', 'mass', ',', 'not', 'from', 'some', 'farcical', 'aquatic', 'ceremony', '.']
```

Notice that our regular expression removed the *s* from *ponds* but also from *is* and *basis*. It produced some non-words like *distribut* and *deriv*, but these are acceptable stems in some applications.

Searching Tokenized Text

You can use a special kind of regular expression for searching across multiple words in a text (where a text is a list of tokens). For example, `"<a> <man>"` finds all instances of *a man* in the text. The angle brackets are used to mark token boundaries, and any whitespace between the angle brackets is ignored (behaviors that are unique to NLTK's `findall()` method for texts). In the following example, we include `<.*>` ❶ which will match any single token, and enclose it in parentheses so only the matched word (e.g. *monied*) and not the matched phrase (e.g. *a monied man*) is produced. The second example finds three-word phrases ending with the word *bro* ❷. The last example finds sequences of three or more words starting with the letter *l* ❸.

```
>>> from nltk.corpus import gutenberg, nps_chat
>>> moby = nltk.Text(gutenberg.words('melville-moby_dick.txt'))
>>> moby.findall(r"<a> (<.*>) <man>") ❶
monied; nervous; dangerous; white; white; white; pious; queer; good;
mature; white; Cape; great; wise; wise; butterless; white; fiendish;
pale; furious; better; certain; complete; dismayed; younger; brave;
brave; brave; brave
>>> chat = nltk.Text(nps_chat.words())
>>> chat.findall(r"<.*> <.*> <bro>") ❷
```

```
you rule bro; telling you bro; u twizted bro
>>> chat.findall(r"<1.*>{3,}")
lol lol lol; lmao lol lol; lol lol lol; la la la la la; la la la; la
la la; lovely lol lol love; lol lol lol.; la la la; la la la
```

Note

Your Turn: Consolidate your understanding of regular expression patterns and substitutions using `nltk.re_show(p, s)` which annotates the string `s` to show every place where pattern `p` was matched, and `nltk.app.nemo()` which provides a graphical interface for exploring regular expressions. For more practice, try some of the exercises on regular expressions at the end of this chapter.

It is easy to build search patterns when the linguistic phenomenon we're studying is tied to particular words. In some cases, a little creativity will go a long way. For instance, searching a large text corpus for expressions of the form *x and other ys* allows us to discover hypernyms (cf [2.5](#)):

```
>>> from nltk.corpus import brown
>>> hobbies_learned = nltk.Text(brown.words(categories=['hobbies', 'learned']))
>>> hobbies_learned.findall(r"<\w*> <and> <other> <\w*s>")
speed and other activities; water and other liquids; tomb and other
landmarks; Statues and other monuments; pearls and other jewels;
charts and other items; roads and other features; figures and other
objects; military and other areas; demands and other factors;
abstracts and other compilations; iron and other metals
```

With enough text, this approach would give us a useful store of information about the taxonomy of objects, without the need for any manual labor. However, our search results will usually contain false positives, i.e. cases that we would want to exclude. For example, the result: *demands and other factors* suggests that *demand* is an instance of the type *factor*, but this sentence is actually about wage demands. Nevertheless, we could construct our own ontology of English concepts by manually correcting the output of such searches.

Note

This combination of automatic and manual processing is the most common way for new corpora to be constructed. We will return to this in [11](#).

Searching corpora also suffers from the problem of false negatives, i.e. omitting cases that we would want to include. It is risky to conclude that some linguistic phenomenon doesn't exist in a corpus just because we couldn't find any instances of a search pattern. Perhaps we just didn't think carefully enough about suitable patterns.

Note

Your Turn: Look for instances of the pattern *as x as y* to discover information about entities and their properties.

3.6 Normalizing Text

In earlier program examples we have often converted text to lowercase before doing anything with its words, e.g. `set(w.lower() for w in text)`. By using `lower()`, we have **normalized** the text to lowercase so that the distinction between *The* and *the* is ignored. Often we want to go further than this, and strip off any affixes, a task known as stemming. A further step is to make sure that the resulting form is a known word in a dictionary, a task known as lemmatization. We discuss each of these in turn. First, we need to define the data we will use in this section:

```
>>> raw = """DENNIS: Listen, strange women lying in ponds distributing swords
... is no basis for a system of government. Supreme executive power derives from
... a mandate from the masses, not from some farcical aquatic ceremony."""
>>> tokens = nltk.word_tokenize(raw)
```

Stemmers

NLTK includes several off-the-shelf stemmers, and if you ever need a stemmer you should use one of these in preference to crafting your own using regular expressions, since these handle a wide range of irregular cases. The Porter and Lancaster stemmers follow their own rules for stripping affixes. Observe that the Porter stemmer correctly handles the word *lying* (mapping it to *lie*), while the Lancaster stemmer does not.

```
>>> porter = nltk.PorterStemmer()
>>> lancaster = nltk.LancasterStemmer()
>>> [porter.stem(t) for t in tokens]
['DENNI', ':', 'Listen', ',', 'strang', 'women', 'lie', 'in', 'pond',
'distribut', 'sword', 'is', 'no', 'basi', 'for', 'a', 'system', 'of', 'govern',
'.', 'Suprem', 'execut', 'power', 'deriv', 'from', 'a', 'mandat', 'from',
'the', 'mass', ',', 'not', 'from', 'some', 'farcic', 'aquat', 'ceremoni', '.']
>>> [lancaster.stem(t) for t in tokens]
['den', ':', 'list', ',', 'strange', 'wom', 'lying', 'in', 'pond', 'distribut',
'sword', 'is', 'no', 'bas', 'for', 'a', 'system', 'of', 'govern', '.', 'suprem',
'execut', 'pow', 'der', 'from', 'a', 'mand', 'from', 'the', 'mass', ',', 'not',
'from', 'som', 'farc', 'aqu', 'ceremony', '.']
```

Stemming is not a well-defined process, and we typically pick the stemmer that best suits the application we have in mind. The Porter Stemmer is a good choice if you are indexing some texts and want to support search using alternative forms of words (illustrated in 3.6, which uses *object oriented* programming techniques that are outside the scope of this book, string formatting techniques to be covered in 3.9, and the `enumerate()` function to be explained in 4.2).

```
class IndexedText(object):

    def __init__(self, stemmer, text):
        self._text = text
        self._stemmer = stemmer
        self._index = nltk.Index((self._stem(word), i)
                                for (i, word) in enumerate(text))

    def concordance(self, word, width=40):
        key = self._stem(word)
        wc = width/4                # words of context
        for i in self._index[key]:
            lcontext = ' '.join(self._text[i-wc:i])
            rcontext = ' '.join(self._text[i:i+wc])
            ldisplay = '%*s' % (width, lcontext[-width:])
            rdisplay = '%-*s' % (width, rcontext[:width])
            print ldisplay, rdisplay

    def _stem(self, word):
        return self._stemmer.stem(word).lower()

>>> porter = nltk.PorterStemmer()
>>> grail = nltk.corpus.webtext.words('grail.txt')
>>> text = IndexedText(porter, grail)
>>> text.concordance('lie')
r king ! DENNIS : Listen , strange women lying in ponds distributing swords is no
beat a very brave retreat . ROBIN : All lies ! MINSTREL : [ singing ] Bravest of
Nay . Nay . Come . Come . You may lie here . Oh , but you are wounded !
doctors immediately ! No , no , please ! Lie down . [ clap clap ] PIGLET : Well
ere is much danger , for beyond the cave lies the Gorge of Eternal Peril , which
you . Oh ... TIM : To the north there lies a cave -- the cave of Caerbannog --
h it and lived ! Bones of full fifty men lie strewn about its lair . So , brave k
not stop our fight ' til each one of you lies dead , and the Holy Grail returns t
```

Example 3.6 (code `stemmer_indexing.py`): Figure 3.6: Indexing a Text Using a Stemmer

Lemmatization

The WordNet lemmatizer only removes affixes if the resulting word is in its dictionary. This additional checking process makes the lemmatizer slower than the above stemmers. Notice that it doesn't handle *lying*, but it converts *women* to *woman*.

```
>>> wn1 = nltk.WordNetLemmatizer()
>>> [wn1.lemmatize(t) for t in tokens]
['DENNIS', ':', 'Listen', ',', 'strange', 'woman', 'lying', 'in', 'pond',
'distributing', 'sword', 'is', 'no', 'basis', 'for', 'a', 'system', 'of',
'government', '.', 'Supreme', 'executive', 'power', 'derives', 'from', 'a',
'mandate', 'from', 'the', 'mass', ',', 'not', 'from', 'some', 'farcical',
'aquatic', 'ceremony', '.']
```

The WordNet lemmatizer is a good choice if you want to compile the vocabulary of some texts and want a list of valid lemmas (or lexicon headwords).

Note

Another normalization task involves identifying **non-standard words** including numbers, abbreviations, and dates, and mapping any such tokens to a special vocabulary. For example, every decimal number could be mapped to a single token `0.0`, and every acronym could be mapped to `AAA`. This keeps the vocabulary small and improves the accuracy of many language modeling tasks.

3.7 Regular Expressions for Tokenizing Text

Tokenization is the task of cutting a string into identifiable linguistic units that constitute a piece of language data. Although it is a fundamental task, we have been able to delay it until now because many corpora are already tokenized, and because NLTK includes some tokenizers. Now that you are familiar with regular expressions, you can learn how to use them to tokenize text, and to have much more control over the process.

Simple Approaches to Tokenization

The very simplest method for tokenizing text is to split on whitespace. Consider the following text from *Alice's Adventures in Wonderland*:

```
>>> raw = """'When I'M a Duchess,' she said to herself, (not in a very hopeful tone
... though), 'I won't have any pepper in my kitchen AT ALL. Soup does very
... well without--Maybe it's always pepper that makes people hot-tempered,...'"""
```

We could split this raw text on whitespace using `raw.split()`. To do the same using a regular expression, it is not enough to match any space characters in the string ❶ since this results in tokens that contain a `\n` newline character; instead we need to match any number of spaces, tabs, or newlines ❷:

```
>>> re.split(r' ', raw) ❶
['When', 'I'M', 'a', 'Duchess,', 'she', 'said', 'to', 'herself,', '(not', 'in',
'a', 'very', 'hopeful', 'tone\nthough)', 'I', 'won't', 'have', 'any', 'pepper',
'in', 'my', 'kitchen', 'AT', 'ALL.', 'Soup', 'does', 'very\nwell', 'without--Maybe',
'it's', 'always', 'pepper', 'that', 'makes', 'people', 'hot-tempered,...']
>>> re.split(r'[ \t\n]+', raw) ❷
['When', 'I'M', 'a', 'Duchess,', 'she', 'said', 'to', 'herself,', '(not', 'in',
'a', 'very', 'hopeful', 'tone', 'though)', 'I', 'won't', 'have', 'any', 'pepper',
'in', 'my', 'kitchen', 'AT', 'ALL.', 'Soup', 'does', 'very', 'well', 'without--Maybe',
'it's', 'always', 'pepper', 'that', 'makes', 'people', 'hot-tempered,...']
```

The regular expression `[ \t\n]+` matches one or more space, tab (`\t`) or newline (`\n`). Other whitespace characters, such as carriage-return and form-feed should really be included too. Instead, we will use a built-in `re` abbreviation, `\s`, which means any whitespace character. The above statement can be rewritten as `re.split(r'\s+', raw)`.

Note

Important: Remember to prefix regular expressions with the letter `r` (meaning "raw"), which instructs the Python interpreter to treat the string literally, rather than processing any backslashed characters it contains.

Splitting on whitespace gives us tokens like `'(not'` and `'herself,'`. An alternative is to use the fact that Python provides us with a character class `\w` for word characters, equivalent to `[a-zA-Z0-9_]`. It also defines the complement of this class `\W`, i.e. all characters other than letters, digits or underscore. We can use `\W` in a simple regular expression to split the input on anything *other* than a word character:

```
>>> re.split(r'\W+', raw)
['', 'When', 'I', 'M', 'a', 'Duchess', 'she', 'said', 'to', 'herself', 'not', 'in',
'a', 'very', 'hopeful', 'tone', 'though', 'I', 'won', 't', 'have', 'any', 'pepper',
'in', 'my', 'kitchen', 'AT', 'ALL', 'Soup', 'does', 'very', 'well', 'without',
'Maybe', 'it', 's', 'always', 'pepper', 'that', 'makes', 'people', 'hot', 'tempered',
'']
```

Observe that this gives us empty strings at the start and the end (to understand why, try doing `'xx'.split('x')`). We get the same tokens,

but without the empty strings, with `re.findall(r'\w+|\S\w*', raw)`, using a pattern that matches the words instead of the spaces. Now that we're matching the words, we're in a position to extend the regular expression to cover a wider range of cases. The regular expression `<<\w+|\S\w*>>` will first try to match any sequence of word characters. If no match is found, it will try to match any *non*-whitespace character (`\S` is the complement of `\s`) followed by further word characters. This means that punctuation is grouped with any following letters (e.g. 's) but that sequences of two or more punctuation characters are separated.

```
>>> re.findall(r'\w+|\S\w*', raw)
['When', 'I', 'M', 'a', 'Duchess', ',', '"', 'she', 'said', 'to', 'herself', ',',
'not', 'in', 'a', 'very', 'hopeful', 'tone', 'though', ')', ',', '"', 'I', 'won', 't',
'have', 'any', 'pepper', 'in', 'my', 'kitchen', 'AT', 'ALL', '.', 'Soup', 'does',
'very', 'well', 'without', '-', '-Maybe', 'it', 's', 'always', 'pepper', 'that',
'makes', 'people', 'hot', '-tempered', ',', '"', '.', '.', '.']
```

Let's generalize the `\w+` in the above expression to permit word-internal hyphens and apostrophes: `<<\w+([-']\w+)*>>`. This expression means `\w+` followed by zero or more instances of `[-']\w+`; it would match *hot-tempered* and *it's*. (We need to include `?` in this expression for reasons discussed earlier.) We'll also add a pattern to match quote characters so these are kept separate from the text they enclose.

```
>>> print re.findall(r"\w+([?:(-']\w+)*|'[-.()'+|\S\w*", raw)
['"', 'When', 'I', 'M', 'a', 'Duchess', ',', '"', 'she', 'said', 'to', 'herself', ',',
'(', 'not', 'in', 'a', 'very', 'hopeful', 'tone', 'though', ')', ',', '"', 'I',
'won't', 'have', 'any', 'pepper', 'in', 'my', 'kitchen', 'AT', 'ALL', '.', 'Soup',
'does', 'very', 'well', 'without', '--', 'Maybe', 'it's', 'always', 'pepper',
'that', 'makes', 'people', 'hot-tempered', ',', '"', '...', '.']
```

The above expression also included `<[-.()'+>` which causes the double hyphen, ellipsis, and open parenthesis to be tokenized separately.

[3.4](#) lists the regular expression character class symbols we have seen in this section, in addition to some other useful symbols.

Table 3.4:

Regular Expression Symbols

Symbol	Function
<code>\b</code>	Word boundary (zero width)
<code>\d</code>	Any decimal digit (equivalent to <code>[0-9]</code>)
<code>\D</code>	Any non-digit character (equivalent to <code>[^0-9]</code>)
<code>\s</code>	Any whitespace character (equivalent to <code>[\t\n\r\f\v]</code>)
<code>\S</code>	Any non-whitespace character (equivalent to <code>[^\t\n\r\f\v]</code>)
<code>\w</code>	Any alphanumeric character (equivalent to <code>[a-zA-Z0-9_]</code>)
<code>\W</code>	Any non-alphanumeric character (equivalent to <code>[^a-zA-Z0-9_]</code>)
<code>\t</code>	The tab character
<code>\n</code>	The newline character

NLTK's Regular Expression Tokenizer

The function `nltk.regexp_tokenize()` is similar to `re.findall()` (as we've been using it for tokenization). However, `nltk.regexp_tokenize()` is more efficient for this task, and avoids the need for special treatment of parentheses. For readability we break up the regular expression over several lines and add a comment about each line. The special `(?x)` "verbose flag" tells Python to strip out the embedded whitespace and comments.

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)
...     ([A-Z]\.)+      # abbreviations, e.g. U.S.A.
...     | \w+(-\w+)*    # words with optional internal hyphens
...     | \$?\d+(\.\d+)?%? # currency and percentages, e.g. $12.40, 82%
...     | \.\.\.        # ellipsis
...     | [!,:;"'()?:_-] # these are separate tokens
... '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

When using the verbose flag, you can no longer use `' '` to match a space character; use `\s` instead. The `regexp_tokenize()` function has an optional `gaps` parameter. When set to `True`, the regular expression specifies the gaps between tokens, as with `re.split()`.

Note

We can evaluate a tokenizer by comparing the resulting tokens with a wordlist, and reporting any tokens that don't appear in the wordlist, using `set(tokens).difference(wordlist)`. You'll probably want to lowercase all the tokens first.

Further Issues with Tokenization

Tokenization turns out to be a far more difficult task than you might have expected. No single solution works well across-the-board, and we must decide what counts as a token depending on the application domain.

When developing a tokenizer it helps to have access to raw text which has been manually tokenized, in order to compare the output of your tokenizer with high-quality (or "gold-standard") tokens. The NLTK corpus collection includes a sample of Penn Treebank data, including the raw Wall Street Journal text (`nltk.corpus.treebank_raw.raw()`) and the tokenized version (`nltk.corpus.treebank.words()`).

A final issue for tokenization is the presence of contractions, such as *didn't*. If we are analyzing the meaning of a sentence, it would probably be more useful to normalize this form to two separate forms: *did* and *n't* (or *not*). We can do this work with the help of a lookup table.

3.8 Segmentation

This section discusses more advanced concepts, which you may prefer to skip on the first time through this chapter.

Tokenization is an instance of a more general problem of **segmentation**. In this section we will look at two other instances of this problem, which use radically different techniques to the ones we have seen so far in this chapter.

Sentence Segmentation

Manipulating texts at the level of individual words often presupposes the ability to divide a text into individual sentences. As we have seen, some corpora already provide access at the sentence level. In the following example, we compute the average number of words per sentence in the Brown Corpus:

```
>>> len(nltk.corpus.brown.words()) / len(nltk.corpus.brown.sents())
20.250994070456922
```

In other cases, the text is only available as a stream of characters. Before tokenizing the text into words, we need to segment it into sentences. NLTK facilitates this by including the Punkt sentence segmenter ([Kiss & Strunk, 2006](#)). Here is an example of its use in segmenting the text of a novel. (Note that if the segmenter's internal data has been updated by the time you read this, you will see different output):

```
>>> sent_tokenizer=nltk.data.load('tokenizers/punkt/english.pickle')
>>> text = nltk.corpus.gutenberg.raw('chesterton-thursday.txt')
>>> sents = sent_tokenizer.tokenize(text)
>>> pprint.pprint(sents[171:181])
["Nonsense!",
 '" said Gregory, who was very rational when anyone else\nattempted paradox.',
 '"Why do all the clerks and navvies in the\nrailway trains look so sad and tired,...',
 'I will\ntell you.',
 'It is because they know that the train is going right.',
 'It\nis because they know that whatever place they have taken a ticket\nfor that ...',
 'It is because after they have\npassed Sloane Square they know that the next stat...',
 'Oh, their wild rapture!',
 'oh,\ntheir eyes like stars and their souls again in Eden, if the next\nstation w...'
 '\n\n'It is you who are unpoetical," replied the poet Syme.']
```

Notice that this example is really a single sentence, reporting the speech of Mr Lucian Gregory. However, the quoted speech contains several sentences, and these have been split into individual strings. This is reasonable behavior for most applications.

Sentence segmentation is difficult because period is used to mark abbreviations, and some periods simultaneously mark an abbreviation and terminate a sentence, as often happens with acronyms like *U.S.A.*

For another approach to sentence segmentation, see [6.2](#).

Word Segmentation

For some writing systems, tokenizing text is made more difficult by the fact that there is no visual representation of word boundaries.

For example, in Chinese, the three-character string: 爱国人 (ai4 "love" (verb), guo3 "country", ren2 "person") could be tokenized as 爱国 / 人, "country-loving person" or as 爱 / 国人, "love country-person."

A similar problem arises in the processing of spoken language, where the hearer must segment a continuous speech stream into individual words. A particularly challenging version of this problem arises when we don't know the words in advance. This is the problem faced by a language learner, such as a child hearing utterances from a parent. Consider the following artificial example, where word boundaries have been removed:

- (1) a. *doyouseethekitty*
b. *seethedoggy*
c. *doyoulikethekitty*
d. *likethedoggy*

Our first challenge is simply to represent the problem: we need to find a way to separate text content from the segmentation. We can do this by annotating each character with a boolean value to indicate whether or not a word-break appears after the character (an idea that will be used heavily for "chunking" in 7). Let's assume that the learner is given the utterance breaks, since these often correspond to extended pauses. Here is a possible representation, including the initial and target segmentations:

```
>>> text = "doyouseethekittyseethedoggydoyoulikethekittylikethedoggy"
>>> seg1 = "00000000000000001000000000001000000000000000000000000"
>>> seg2 = "0100100100100001001001000010100100010010000100010010000"
```

Observe that the segmentation strings consist of zeros and ones. They are one character shorter than the source text, since a text of length n can only be broken up in $n-1$ places. The `segment()` function in [3.7](#) demonstrates that we can get back to the original segmented text from the above representation.

```
def segment(text, segs):
    words = []
    last = 0
    for i in range(len(segs)):
        if segs[i] == '1':
            words.append(text[last:i+1])
            last = i+1
    words.append(text[last:])
    return words

>>> text = "doyouseethekittyseethedoggydoyoulikethekittylikethedoggy"
>>> seg1 = "000000000000000010000000000010000000000000000100000000000"
>>> seg2 = "01001001001000010010010000101001000100100001000100010000"
>>> segment(text, seg1)
['doyouseethekitty', 'seethedoggy', 'doyoulikethekitty', 'likethedoggy']
>>> segment(text, seg2)
['do', 'you', 'see', 'the', 'kitty', 'see', 'the', 'doggy', 'do', 'you', 'like', 'the', 'kitty', 'like', 'the', 'doggy']
```

Example 3.7 (code_segment.py): Figure 3.7: Reconstruct Segmented Text from String Representation: `seg1` and `seg2` represent the initial and final segmentations of some hypothetical child-directed speech; the `segment()` function can use them to reproduce the segmented text.

Now the segmentation task becomes a search problem: find the bit string that causes the text string to be correctly segmented into words. We assume the learner is acquiring words and storing them in an internal lexicon. Given a suitable lexicon, it is possible to reconstruct the source text as a sequence of lexical items. Following (Brent, 1995), we can define an **objective function**, a scoring function whose value we will try to optimize, based on the size of the lexicon and the amount of information needed to reconstruct the source text from the lexicon. We illustrate this in 3.8.

SEGMENTATION	REPRESENTATION	OBJECTIVE
	LEXICON	DERIVATION
doyou see thekitt y	1. doyou	1 2 4 6
see thedogg y	2. see	2 5 6
doyou like thekitt y	3. like	2 5 6
	4. thekitt	1 3 4 6
	5. thedogg	1 3 4 6
like thedogg y	6. y	3 5 6
		LEXICON: $6+4+5+8+8+2 = 33$
		DERIVATION: $4+3+4+3 = 14$
		TOTAL: $33+14 = 47$

Figure 3.8: Calculation of Objective Function: Given a hypothetical segmentation of the source text (on the left), derive a lexicon and a derivation table that permit the source text to be reconstructed, then total up the number of characters used by each lexical item (including a boundary marker) and each derivation, to serve as a score of the quality of the segmentation; smaller values of the score indicate a better segmentation.

It is a simple matter to implement this objective function, as shown in [3.9](#).

```
def evaluate(text, segs):
    words = segment(text, segs)
    text_size = len(words)
    lexicon_size = len(' '.join(list(set(words))))
    return text_size + lexicon_size

>>> text = "doyouseethekittyseethedoggydoyoulikethekittylikethedoggy"
>>> seg1 = "0000000000000000100000000000100000000000000100000000000"
>>> seg2 = "0100100100100001001001000010100100010010000100010010000"
>>> seg3 = "0000100100000011001000000110000100010000001100010000001"
>>> segment(text, seg3)
['doyou', 'see', 'thekitt', 'y', 'see', 'thedogg', 'y', 'doyou', 'like',
 'thekitt', 'y', 'like', 'thedogg', 'y']
>>> evaluate(text, seg3)
46
>>> evaluate(text, seg2)
47
>>> evaluate(text, seg1)
63
```

Example 3.9 (code_evaluate.py): **Figure 3.9:** Computing the Cost of Storing the Lexicon and Reconstructing the Source Text

The final step is to search for the pattern of zeros and ones that minimizes this objective function, shown in [3.10](#). Notice that the best segmentation includes "words" like *thekitty*, since there's not enough evidence in the data to split this any further.

```

from random import randint

def flip(segs, pos):
    return segs[:pos] + str(1-int(segs[pos])) + segs[pos+1:]

def flip_n(segs, n):
    for i in range(n):
        segs = flip(segs, randint(0,len(segs)-1))
    return segs

def anneal(text, segs, iterations, cooling_rate):
    temperature = float(len(segs))
    while temperature > 0.5:
        best_segs, best = segs, evaluate(text, segs)
        for i in range(iterations):
            guess = flip_n(segs, int(round(temperature)))
            score = evaluate(text, guess)
            if score < best:
                best, best_segs = score, guess
        score, segs = best, best_segs
        temperature = temperature / cooling_rate
    print evaluate(text, segs), segment(text, segs)
    print
    return segs

>>> text = "dovouseethekittyseethedoggydovoulikethekittylikedoggy"

```

```
>>> seg1 = "00000000000000000000100000000000100000000000000001000000000000"
>>> anneal(text, seg1, 5000, 1.2)
60 ['doyouseetheki', 'tty', 'see', 'thedoggy', 'doyouliketh', 'ekittylike', 'thedoggy']
58 ['doy', 'ouseetheki', 'ttysee', 'thedoggy', 'doy', 'o', 'ulikethekittylike', 'thedoggy']
56 ['doyou', 'seetheki', 'ttysee', 'thedoggy', 'doyou', 'liketh', 'ekittylike', 'thedoggy']
54 ['doyou', 'seethekit', 'tysee', 'thedoggy', 'doyou', 'likethekittylike', 'thedoggy']
53 ['doyou', 'seethekit', 'tysee', 'thedoggy', 'doyou', 'like', 'thekitty', 'like', 'thedoggy']
51 ['doyou', 'seethekittysee', 'thedoggy', 'doyou', 'like', 'thekitty', 'like', 'thedoggy']
42 ['doyou', 'see', 'thekitty', 'see', 'thedoggy', 'doyou', 'like', 'thekitty', 'like', 'thedoggy']
'0000100100000000100100000000100000100000000100010000000'
```

Example 3.10 (code_anneal.py): Figure 3.10: Non-Deterministic Search Using Simulated Annealing: begin searching with phrase segmentations only; randomly perturb the zeros and ones proportional to the "temperature"; with each iteration the temperature is lowered and the perturbation of boundaries is reduced.

With enough data, it is possible to automatically segment text into words with a reasonable degree of accuracy. Such methods can be applied to tokenization for writing systems that don't have any visual representation of word boundaries.

3.9 Formatting: From Lists to Strings

Often we write a program to report a single data item, such as a particular element in a corpus that meets some complicated criterion, or a single summary statistic such as a word-count or the performance of a tagger. More often, we write a program to produce a structured result; for example, a tabulation of numbers or linguistic forms, or a reformatting of the original data. When the results to be presented are linguistic, textual output is usually the most natural choice. However, when the results are numerical, it may be preferable to produce graphical output. In this section you will learn about a variety of ways to present program output.

From Lists to Strings

The simplest kind of structured object we use for text processing is lists of words. When we want to output these to a display or a file, we must convert these lists into strings. To do this in Python we use the `join()` method, and specify the string to be used as the "glue".

```
>>> silly = ['We', 'called', 'him', 'Tortoise', 'because', 'he', 'taught', 'us', '.']
>>> ' '.join(silly)
'We called him Tortoise because he taught us .'
>>> ';'.join(silly)
'We;called;him;Tortoise;because;he;taught;us;.'
>>> ''.join(silly)
'WecalledhimTortoisebecausehetaughtus.'
```

So `' '.join(silly)` means: take all the items in `silly` and concatenate them as one big string, using `' '` as a spacer between the items. I.e. `join()` is a method of the string that you want to use as the glue. (Many people find this notation for `join()` counter-intuitive.) The `join()` method only works on a list of strings — what we have been calling a text — a complex type that enjoys some privileges in Python.

Strings and Formats

We have seen that there are two ways to display the contents of an object:

```
>>> word = 'cat'
>>> sentence = """hello
... world"""
>>> print word
cat
>>> print sentence
hello
world
>>> word
'cat'
>>> sentence
'hello\nworld'
```

The `print` command yields Python's attempt to produce the most human-readable form of an object. The second method — naming the variable at a prompt — shows us a string that can be used to recreate this object. It is important to keep in mind that both of these are just strings, displayed for the benefit of you, the user. They do not give us any clue as to the actual internal representation of the object.

There are many other useful ways to display an object as a string of characters. This may be for the benefit of a human reader, or because we want to **export** our data to a particular file format for use in an external program.

Formatted output typically contains a combination of variables and pre-specified strings, e.g. given a frequency distribution `fdist` we

could do:

```
>>> fdist = nltk.FreqDist(['dog', 'cat', 'dog', 'cat', 'dog', 'snake', 'dog', 'cat'])
>>> for word in fdist:
...     print word, '->', fdist[word], ';'
dog -> 4 ; cat -> 3 ; snake -> 1 ;
```

Apart from the problem of unwanted whitespace, print statements that contain alternating variables and constants can be difficult to read and maintain. A better solution is to use **string formatting expressions**.

```
>>> for word in fdist:
...     print '%s->%d;' % (word, fdist[word]),
dog->4; cat->3; snake->1;
```

To understand what is going on here, let's test out the string formatting expression on its own. (By now this will be your usual method of exploring new syntax).

```
>>> '%s->%d;' % ('cat', 3)
'cat->3;'
>>> '%s->%d;' % 'cat'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: not enough arguments for format string
```

The special symbols %s and %d are placeholders for strings and (decimal) integers. We can embed these inside a string, then use the % operator to combine them. Let's unpack the above code further, in order to see this behavior up close:

```
>>> '%s->' % 'cat'
'cat->'
>>> '%d' % 3
'3'
>>> 'I want a %s right now' % 'coffee'
'I want a coffee right now'
```

We can have a number of placeholders, but following the % operator we need to specify a tuple with exactly the same number of values.

```
>>> "%s wants a %s %s" % ("Lee", "sandwich", "for lunch")
'Lee wants a sandwich for lunch'
```

We can also provide the values for the placeholders indirectly. Here's an example using a **for** loop:

```
>>> template = 'Lee wants a %s right now'
>>> menu = ['sandwich', 'spam fritter', 'pancake']
>>> for snack in menu:
...     print template % snack
...
Lee wants a sandwich right now
Lee wants a spam fritter right now
Lee wants a pancake right now
```

The %s and %d symbols are called **conversion specifiers**. They start with the % character and end with a conversion character such as s (for string) or d (for decimal integer). The string containing conversion specifiers is called a **format string**. We combine a format string with the % operator and a tuple of values to create a complete string formatting expression.

Lining Things Up

So far our formatting strings generated output of arbitrary width on the page (or screen), such as %s and %d. We can specify a width as well, such as %6s, producing a string that is padded to width 6. It is right-justified by default ❶, but we can include a minus sign to make it left-justified ❷. In case we don't know in advance how wide a displayed value should be, the width value can be replaced with a star in the formatting string, then specified using a variable ❸.

```
>>> '%6s' % 'dog' ❶
'   dog'
>>> '%-6s' % 'dog' ❷
'dog   '
>>> width = 6
>>> '%-*s' % (width, 'dog') ❸
'dog   '
```

Other control characters are used for decimal integers and floating point numbers. Since the percent character % has a special interpretation in formatting strings, we have to precede it with another % to get it in the output.

```
>>> count, total = 3205, 9375
>>> "accuracy for %d words: %2.4f%%" % (total, 100 * count / total)
'accuracy for 9375 words: 34.1867%'
```

An important use of formatting strings is for tabulating data. Recall that in [2.1](#) we saw data being tabulated from a conditional frequency distribution. Let's perform the tabulation ourselves, exercising full control of headings and column widths, as shown in [3.11](#). Note the clear separation between the language processing work, and the tabulation of results.

```
def tabulate(cfdist, words, categories):
    print '%-16s' % 'Category',
    for word in words:
        print '%6s' % word,
    print
    for category in categories:
        print '%-16s' % category,
        for word in words:
            print '%6d' % cfdist[category][word],
        print

>>> from nltk.corpus import brown
>>> cfd = nltk.ConditionalFreqDist(
...     (genre, word)
...     for genre in brown.categories()
...     for word in brown.words(categories=genre))
>>> genres = ['news', 'religion', 'hobbies', 'science_fiction', 'romance', 'humor']
>>> modals = ['can', 'could', 'may', 'might', 'must', 'will']
>>> tabulate(cfd, modals, genres)
Category          can  could   may  might   must   will
news              93    86    66    38    50   389
religion          82    59    78    12    54    71
hobbies          268    58   131    22    83   264
science_fiction   16    49    4    12    8    16
romance           74   193   11   51   45   43
humor             16    30    8    8    9    13
```

Example 3.11 (code_modal_tabulate.py): Figure 3.11: Frequency of Modals in Different Sections of the Brown Corpus

Recall from the listing in [3.6](#) that we used a formatting string "%*s". This allows us to specify the width of a field using a variable.

```
>>> '%*s' % (15, "Monty Python")
'   Monty Python'
```

We could use this to automatically customize the column to be just wide enough to accommodate all the words, using `width = max(len(w) for w in words)`. Remember that the comma at the end of print statements adds an extra space, and this is sufficient to prevent the column headings from running into each other.

Writing Results to a File

We have seen how to read text from files ([3.1](#)). It is often useful to write output to files as well. The following code opens a file `output.txt` for writing, and saves the program output to the file.

```
>>> output_file = open('output.txt', 'w')
>>> words = set(nltk.corpus.genesis.words('english-kjv.txt'))
>>> for word in sorted(words):
...     output_file.write(word + "\n")
```

Note

Your Turn: What is the effect of appending `\n` to each string before we write it to the file? If you're using a Windows machine you may want to use `word + "\r\n"` instead. What happens if we do `output_file.write(word)`?

```
>>> len(words)
```

```
2789
>>> str(len(words))
'2789'
>>> output_file.write(str(len(words)) + "\n")
>>> output_file.close()
```

Caution!

You should avoid filenames that contain space characters like `output file.txt`, or that are identical except for case distinctions, e.g. `Output.txt` and `output.TXT`.

Text Wrapping

When the output of our program is text-like, instead of tabular, it will usually be necessary to wrap it so that it can be displayed conveniently. Consider the following output, which overflows its line, and which uses a complicated `print` statement:

```
>>> saying = ['After', 'all', 'is', 'said', 'and', 'done', ',',
...           'more', 'is', 'said', 'than', 'done', '.']
>>> for word in saying:
...     print word, '(' + str(len(word)) + '), '
After (5), all (3), is (2), said (4), and (3), done (4), , (1), more (4), is (2), said (4), than (4), done (4), . (1),
```

We can take care of line wrapping with the help of Python's `textwrap` module. For maximum clarity we will separate each step onto its own line:

```
>>> from textwrap import fill
>>> format = '%s (%d),'
>>> pieces = [format % (word, len(word)) for word in saying]
>>> output = ' '.join(pieces)
>>> wrapped = fill(output)
>>> print wrapped
After (5), all (3), is (2), said (4), and (3), done (4), , (1), more
(4), is (2), said (4), than (4), done (4), . (1),
```

Notice that there is a linebreak between `more` and its following number. If we wanted to avoid this, we could redefine the formatting string so that it contained no spaces, e.g. `'%s_(%d),'`, then instead of printing the value of `wrapped`, we could print `wrapped.replace('_', '')`.

3.10 Summary

- In this book we view a text as a list of words. A "raw text" is a potentially long string containing words and whitespace formatting, and is how we typically store and visualize a text.
- A string is specified in Python using single or double quotes: `'Monty Python'`, `"Monty Python"`.
- The characters of a string are accessed using indexes, counting from zero: `'Monty Python'[0]` gives the value `M`. The length of a string is found using `len()`.
- Substrings are accessed using slice notation: `'Monty Python'[1:5]` gives the value `onty`. If the start index is omitted, the substring begins at the start of the string; if the end index is omitted, the slice continues to the end of the string.
- Strings can be split into lists: `'Monty Python'.split()` gives `['Monty', 'Python']`. Lists can be joined into strings: `'/'.join(['Monty', 'Python'])` gives `'Monty/Python'`.
- We can read text from a file `f` using `text = open(f).read()`. We can read text from a URL `u` using `text = urlopen(u).read()`. We can iterate over the lines of a text file using `for line in open(f)`.
- Texts found on the web may contain unwanted material (such as headers, footers, markup), that need to be removed before we do any linguistic processing.
- Tokenization is the segmentation of a text into basic units — or tokens — such as words and punctuation. Tokenization based on whitespace is inadequate for many applications because it bundles punctuation together with words. NLTK provides an off-the-shelf tokenizer `nltk.word_tokenize()`.
- Lemmatization is a process that maps the various forms of a word (such as *appeared*, *appears*) to the canonical or citation form of the word, also known as the lexeme or lemma (e.g. `APPEAR`).
- Regular expressions are a powerful and flexible method of specifying patterns. Once we have imported the `re` module, we can use `re.findall()` to find all substrings in a string that match a pattern.
- If a regular expression string includes a backslash, you should tell Python not to preprocess the string, by using a raw string with an `r` prefix: `r'regexp'`.
- When backslash is used before certain characters, e.g. `\n`, this takes on a special meaning (newline character); however, when backslash is used before regular expression wildcards and operators, e.g. `\.`, `\|`, `\$`, these characters *lose* their special meaning and are matched literally.
- A string formatting expression template `% arg_tuple` consists of a format string template that contains conversion specifiers like `%-6s` and `%0.2d`.

3.11 Further Reading

Extra materials for this chapter are posted at <http://www.nltk.org/>, including links to freely available resources on the web. Remember to consult the Python reference materials at <http://docs.python.org/>. (For example, this documentation covers "universal newline support," explaining how to work with the different newline conventions used by various operating systems.)

For more examples of processing words with NLTK, see the tokenization, stemming and corpus HOWTOs at <http://www.nltk.org/howto>. Chapters 2 and 3 of ([Jurafsky & Martin, 2008](#)) contain more advanced material on regular expressions and morphology. For more extensive discussion of text processing with Python see ([Mertz, 2003](#)). For information about normalizing non-standard words see ([Sproat et al. 2001](#))

There are many references for regular expressions, both practical and theoretical. For an introductory tutorial to using regular expressions in Python, see Kuchling's *Regular Expression HOWTO*, <http://www.amk.ca/python/howto/regex/>. For a comprehensive and detailed manual in using regular expressions, covering their syntax in most major programming languages, including Python, see ([Friedl, 2002](#)). Other presentations include Section 2.1 of ([Jurafsky & Martin, 2008](#)), and Chapter 3 of ([Mertz, 2003](#)).

There are many online resources for Unicode. Useful discussions of Python's facilities for handling Unicode are:

- PEP-100 <http://www.python.org/dev/peps/pep-0100/>
- Jason Orendorff, *Unicode for Programmers*, <http://www.jorendorff.com/articles/unicode/>
- A. M. Kuchling, *Unicode HOWTO*, <http://www.amk.ca/python/howto/unicode>
- Frederik Lundh, *Python Unicode Objects*, <http://effbot.org/zone/unicode-objects.htm>
- Joel Spolsky, *The Absolute Minimum Every Software Developer Absolutely, Positively Must Know About Unicode and Character Sets (No Excuses!)*, <http://www.joelonsoftware.com/articles/Unicode.html>

The problem of tokenizing Chinese text is a major focus of SIGHAN, the ACL Special Interest Group on Chinese Language Processing <http://sighan.org/>. Our method for segmenting English text follows ([Brent, 1995](#)); this work falls in the area of language acquisition ([Niyogi, 2006](#)).

Collocations are a special case of multiword expressions. A **multiword expression** is a small phrase whose meaning and other properties cannot be predicted from its words alone, e.g. *part of speech* ([Baldwin & Kim, 2010](#)).

Simulated annealing is a heuristic for finding a good approximation to the optimum value of a function in a large, discrete search space, based on an analogy with annealing in metallurgy. The technique is described in many Artificial Intelligence texts.

The approach to discovering hyponyms in text using search patterns like *x and other ys* is described by ([Hearst, 1992](#)).

3.12 Exercises

1. ☼ Define a string `s = 'colorless'`. Write a Python statement that changes this to "colourless" using only the slice and concatenation operations.
2. ☼ We can use the slice notation to remove morphological endings on words. For example, `'dogs'[:-1]` removes the last character of `dogs`, leaving `dog`. Use slice notation to remove the affixes from these words (we've inserted a hyphen to indicate the affix boundary, but omit this from your strings): `dish-es`, `run-ning`, `nation-ality`, `un-do`, `pre-heat`.
3. ☼ We saw how we can generate an `IndexError` by indexing beyond the end of a string. Is it possible to construct an index that goes too far to the left, before the start of the string?
4. ☼ We can specify a "step" size for the slice. The following returns every second character within the slice: `monty[6:11:2]`. It also works in the reverse direction: `monty[10:5:-2]` Try these for yourself, then experiment with different step values.
5. ☼ What happens if you ask the interpreter to evaluate `monty[::-1]`? Explain why this is a reasonable result.
6. ☼ Describe the class of strings matched by the following regular expressions.

1. `[a-zA-Z]+`
2. `[A-Z][a-z]*`
3. `p[aeiou]{,2}t`
4. `\d+(\.\d+)?`
5. `([^\aeiou][aeiou][^\aeiou])*`
6. `\w+|^[^\w\s]+`

Test your answers using `nltk.re_show()`.

7. ☼ Write regular expressions to match the following classes of strings:

1. A single determiner (assume that *a*, *an*, and *the* are the only determiners).
 2. An arithmetic expression using integers, addition, and multiplication, such as $2*3+8$.
8. ☼ Write a utility function that takes a URL as its argument, and returns the contents of the URL, with all HTML markup removed. Use `urllib.urlopen` to access the contents of the URL, e.g. `raw_contents = urllib.urlopen('http://www.nltk.org/').read()`.
 9. ☼ Save some text into a file `corpus.txt`. Define a function `load(f)` that reads from the file named in its sole argument, and returns a string containing the text of the file.
 1. Use `nltk.regexp_tokenize()` to create a tokenizer that tokenizes the various kinds of punctuation in this text. Use one multi-line regular expression, with inline comments, using the verbose flag (`?x`).
 2. Use `nltk.regexp_tokenize()` to create a tokenizer that tokenizes the following kinds of expression: monetary amounts; dates; names of people and organizations.
 10. ☼ Rewrite the following loop as a list comprehension:

```
>>> sent = ['The', 'dog', 'gave', 'John', 'the', 'newspaper']
>>> result = []
>>> for word in sent:
...     word_len = (word, len(word))
...     result.append(word_len)
>>> result
[('The', 3), ('dog', 3), ('gave', 4), ('John', 4), ('the', 3), ('newspaper', 9)]
```

11. ☼ Define a string `raw` containing a sentence of your own choosing. Now, split `raw` on some character other than space, such as `'s'`.
12. ☼ Write a `for` loop to print out the characters of a string, one per line.
13. ☼ What is the difference between calling `split` on a string with no argument or with `' '` as the argument, e.g. `sent.split()` versus `sent.split(' ')`? What happens when the string being split contains tab characters, consecutive space characters, or a sequence of tabs and spaces? (In IDLE you will need to use `'\t'` to enter a tab character.)
14. ☼ Create a variable `words` containing a list of words. Experiment with `words.sort()` and `sorted(words)`. What is the difference?
15. ☼ Explore the difference between strings and integers by typing the following at a Python prompt: `"3" * 7` and `3 * 7`. Try converting between strings and integers using `int("3")` and `str(3)`.
16. ☼ Earlier, we asked you to use a text editor to create a file called `test.py`, containing the single line `monty = 'Monty Python'`. If you haven't already done this (or can't find the file), go ahead and do it now. Next, start up a new session with the Python interpreter, and enter the expression `monty` at the prompt. You will get an error from the interpreter. Now, try the following (note that you have to leave off the `.py` part of the filename):

```
>>> from test import monty
>>> monty
```

This time, Python should return with a value. You can also try `import test`, in which case Python should be able to evaluate the expression `test.monty` at the prompt.

17. ☼ What happens when the formatting strings `%s` and `%-6s` are used to display strings that are longer than six characters?
18. • Read in some text from a corpus, tokenize it, and print the list of all *wh*-word types that occur. (*wh*-words in English are used in questions, relative clauses and exclamations: *who*, *which*, *what*, and so on.) Print them in order. Are any words duplicated in this list, because of the presence of case distinctions or punctuation?
19. • Create a file consisting of words and (made up) frequencies, where each line consists of a word, the space character, and a positive integer, e.g. `fuzzy 53`. Read the file into a Python list using `open(filename).readlines()`. Next, break each line into its two fields using `split()`, and convert the number into an integer using `int()`. The result should be a list of the form: `[['fuzzy', 53], ...]`.
20. • Write code to access a favorite webpage and extract some text from it. For example, access a weather site and extract the forecast top temperature for your town or city today.
21. • Write a function `unknown()` that takes a URL as its argument, and returns a list of unknown words that occur on that webpage. In order to do this, extract all substrings consisting of lowercase letters (using `re.findall()`) and remove any items from this set that occur in the Words Corpus (`nltk.corpus.words`). Try to categorize these words manually and discuss your findings.
22. • Examine the results of processing the URL `http://news.bbc.co.uk/` using the regular expressions suggested above. You will see

that there is still a fair amount of non-textual data there, particularly Javascript commands. You may also find that sentence breaks have not been properly preserved. Define further regular expressions that improve the extraction of text from this web page.

23. • Are you able to write a regular expression to tokenize text in such a way that the word *don't* is tokenized into *do* and *n't*? Explain why this regular expression won't work: `<n't|\w+>`.
24. • Try to write code to convert text into *hAck3r*, using regular expressions and substitution, where $e \rightarrow 3$, $i \rightarrow 1$, $o \rightarrow 0$, $l \rightarrow |$, $s \rightarrow 5$, $. \rightarrow 5w33t!$, $ate \rightarrow 8$. Normalize the text to lowercase before converting it. Add more substitutions of your own. Now try to map *s* to two different values: $\$$ for word-initial *s*, and 5 for word-internal *s*.
25. • *Pig Latin* is a simple transformation of English text. Each word of the text is converted as follows: move any consonant (or consonant cluster) that appears at the start of the word to the end, then append *ay*, e.g. *string* $\rightarrow$ *ingstray*, *idle* $\rightarrow$ *idleay*.
http://en.wikipedia.org/wiki/Pig_Latin
 1. Write a function to convert a word to Pig Latin.
 2. Write code that converts text, instead of individual words.
 3. Extend it further to preserve capitalization, to keep *qu* together (i.e. so that *quiet* becomes *ietquay*), and to detect when *y* is used as a consonant (e.g. *yellow*) vs a vowel (e.g. *style*).
26. • Download some text from a language that has vowel harmony (e.g. Hungarian), extract the vowel sequences of words, and create a vowel bigram table.
27. • Python's `random` module includes a function `choice()` which randomly chooses an item from a sequence, e.g. `choice("aeHH ")` will produce one of four possible characters, with the letter *h* being twice as frequent as the others. Write a generator expression that produces a sequence of 500 randomly chosen letters drawn from the string `"aeHH "`, and put this expression inside a call to the `''.join()` function, to concatenate them into one long string. You should get a result that looks like uncontrolled sneezing or maniacal laughter: `he haha ee heheeh eha`. Use `split()` and `join()` again to normalize the whitespace in this string.
28. • Consider the numeric expressions in the following sentence from the MedLine Corpus: *The corresponding free cortisol fractions in these sera were 4.53 +/- 0.15% and 8.16 +/- 0.23%, respectively*. Should we say that the numeric expression *4.53 +/- 0.15%* is three words? Or should we say that it's a single compound word? Or should we say that it is actually *nine* words, since it's read "four point five three, plus or minus fifteen percent"? Or should we say that it's not a "real" word at all, since it wouldn't appear in any dictionary? Discuss these different possibilities. Can you think of application domains that motivate at least two of these answers?
29. • Readability measures are used to score the reading difficulty of a text, for the purposes of selecting texts of appropriate difficulty for language learners. Let us define μ_w to be the average number of letters per word, and μ_s to be the average number of words per sentence, in a given text. The Automated Readability Index (ARI) of the text is defined to be: $4.71 \mu_w + 0.5 \mu_s - 21.43$. Compute the ARI score for various sections of the Brown Corpus, including section *f* (popular lore) and *j* (learned). Make use of the fact that `nltk.corpus.brown.words()` produces a sequence of words, while `nltk.corpus.brown.sents()` produces a sequence of sentences.
30. • Use the Porter Stemmer to normalize some tokenized text, calling the stemmer on each word. Do the same thing with the Lancaster Stemmer and see if you observe any differences.
31. • Define the variable `saying` to contain the list `['After', 'all', 'is', 'said', 'and', 'done', ',', 'more', 'is', 'said', 'than', 'done', '.']`. Process this list using a `for` loop, and store the result in a new list `lengths`. Hint: begin by assigning the empty list to `lengths`, using `lengths = []`. Then each time through the loop, use `append()` to add another length value to the list.
32. • Define a variable `silly` to contain the string: `'newly formed bland ideas are inexpressible in an infuriating way'`. (This happens to be the legitimate interpretation that bilingual English-Spanish speakers can assign to Chomsky's famous nonsense phrase, *colorless green ideas sleep furiously* according to Wikipedia). Now write code to perform the following tasks:
 1. Split `silly` into a list of strings, one per word, using Python's `split()` operation, and save this to a variable called `bland`.
 2. Extract the second letter of each word in `silly` and join them into a string, to get `'eoldrnnnna'`.
 3. Combine the words in `bland` back into a single string, using `join()`. Make sure the words in the resulting string are separated with whitespace.
 4. Print the words of `silly` in alphabetical order, one per line.
33. • The `index()` function can be used to look up items in sequences. For example, `'inexpressible'.index('e')` tells us the index of the first position of the letter *e*.
 1. What happens when you look up a substring, e.g. `'inexpressible'.index('re')`?
 2. Define a variable `words` containing a list of words. Now use `words.index()` to look up the position of an individual word.
 3. Define a variable `silly` as in the exercise above. Use the `index()` function in combination with list slicing to build a list

phrase consisting of all the words up to (but not including) **in** in **silly**.

34. • Write code to convert nationality adjectives like *Canadian* and *Australian* to their corresponding nouns *Canada* and *Australia* (see http://en.wikipedia.org/wiki/List_of_adjectival_forms_of_place_names).
35. • Read the LanguageLog post on phrases of the form *as best as p can* and *as best p can*, where *p* is a pronoun. Investigate this phenomenon with the help of a corpus and the `findall()` method for searching tokenized text described in [3.5](#).
<http://itre.cis.upenn.edu/~myl/language-log/archives/002733.html>
36. • Study the *lolcat* version of the book of Genesis, accessible as `nltk.corpus.genesis.words('lolcat.txt')`, and the rules for converting text into *lolspeak* at http://www.lolcatbible.com/index.php?title=How_to_speak_lolcat. Define regular expressions to convert English words into corresponding *lolspeak* words.
37. • Read about the `re.sub()` function for string substitution using regular expressions, using `help(re.sub)` and by consulting the further readings for this chapter. Use `re.sub` in writing code to remove HTML tags from an HTML file, and to normalize whitespace.
38. ★ An interesting challenge for tokenization is words that have been split across a line-break. E.g. if *long-term* is split, then we have the string `long-\nterm`.
 1. Write a regular expression that identifies words that are hyphenated at a line-break. The expression will need to include the `\n` character.
 2. Use `re.sub()` to remove the `\n` character from these words.
 3. How might you identify words that should not remain hyphenated once the newline is removed, e.g. `'encyclo-\npedia'`.
39. ★ Read the Wikipedia entry on *Soundex*. Implement this algorithm in Python.
40. ★ Obtain raw texts from two or more genres and compute their respective reading difficulty scores as in the earlier exercise on reading difficulty. E.g. compare ABC Rural News and ABC Science News (`nltk.corpus.abc`). Use `Punkt` to perform sentence segmentation.
41. ★ Rewrite the following nested loop as a nested list comprehension:

```
>>> words = ['attribution', 'confabulation', 'elocution',
...          'sequoia', 'tenacious', 'unidirectional']
>>> vsequences = set()
>>> for word in words:
...     vowels = []
...     for char in word:
...         if char in 'aeiou':
...             vowels.append(char)
...     vsequences.add(''.join(vowels))
>>> sorted(vsequences)
['aiuiou', 'eauiou', 'euiou', 'euoia', 'ouauiou', 'uiieioa']
```

42. ★ Use WordNet to create a semantic index for a text collection. Extend the concordance search program in [3.6](#), indexing each word using the offset of its first synset, e.g. `wn.synsets('dog')[0].offset` (and optionally the offset of some of its ancestors in the hypernym hierarchy).
43. ★ With the help of a multilingual corpus such as the Universal Declaration of Human Rights Corpus (`nltk.corpus.udhr`), and NLTK's frequency distribution and rank correlation functionality (`nltk.FreqDist`, `nltk.spearman_correlation`), develop a system that guesses the language of a previously unseen text. For simplicity, work with a single character encoding and just a few languages.
44. ★ Write a program that processes a text and discovers cases where a word has been used with a novel sense. For each word, compute the WordNet similarity between all synsets of the word and all synsets of the words in its context. (Note that this is a crude approach; doing it well is a difficult, open research problem.)
45. ★ Read the article on normalization of non-standard words ([Sproat et al. 2001](#)), and implement a similar system for text normalization.

About this document...

This is a chapter from *Natural Language Processing with Python*, by [Steven Bird](#), [Ewan Klein](#) and [Edward Loper](#). Copyright © 2009 the authors. It is distributed with the *Natural Language Toolkit* [<http://www.nltk.org/>], Version 2.0.1rc1, under the terms of the *Creative Commons Attribution-Noncommercial-No Derivative Works 3.0 United States License* [<http://creativecommons.org/licenses/by-nc-nd/3.0/us/>].

This document was built on Mon 15 Oct 2012 16:46:09 EST